Universidad Tecnológica del Perú

Facultad de Ingeniería

AVANCE DE PROYECTO FINAL 3

AYNI BANK · Banca 100 % digital

Curso:
Integrador II: Software

Docente:
Dr. Carlos R. P. Tovar

Grupo:
01 · sección 35362

Integrantes:

1. Fabian García Champi — Cod. U21201692
2. Joaquin Alfonso Loa Denegri — Cod. U22234069
3. Kiara Mishell Santti Saavedra — Cod. U22201636
4. Gerardo Raul Socualaya Mandamiento — Cod. U22229068
5. Frank Grheg Sotomayor Suasnabar — Cod. U18208457
6. Eduardo Vargas Zumaeta — Cod. U22234544

Lima, Lunes 02 de noviembre del 2026

Índice

INTRODUCCIÓN 4

Cómo leer este documento 4

Enlaces del proyecto 4

1. LEVANTAMIENTO DE OBSERVACIONES DEL APF2 5

2. CALIDAD FUNCIONAL Y PRUEBAS AUTOMATIZADAS (CAP. 12) 10

2.1 Frameworks utilizados 10

2.2 Listado de casos de prueba 10

2.3 Evidencia de ejecución 11

2.4 Reportes de cobertura 20

2.5 Métricas, nivel de cumplimiento y observaciones 24

3. INTEROPERABILIDAD Y PRUEBAS DE INTEGRACIÓN (CAP. 13) 25

3.1 Sistemas externos a integrar 25

3.2 Frameworks utilizados 25

3.3 Lista de casos de prueba 26

3.4 Evidencia de ejecución 28

3.5 Reporte de resultados 31

3.6 Métricas, nivel de cumplimiento y observaciones 31

4. PRUEBAS AUTOMATIZADAS DE USABILIDAD (CAP. 14) 32

4.1 Frameworks utilizados 32

4.2 Lista de casos de prueba 32

4.3 Evidencia de ejecución 34

4.4 Reportes de ejecución 36

4.5 Métricas, nivel de cumplimiento y observaciones 36

5. INFORME DE EVALUACIÓN DE USABILIDAD SEGÚN ISO/IEC 25010 (CAP. 15) 37

5.1 Criterios, métricas y evidencias empleadas 37

5.1.1 Facilidad de aprendizaje 37

5.1.2 Protección contra errores del usuario 39

5.1.3 Asistencia al usuario 42

5.1.4 Compromiso del usuario 45

5.2 Resultados 48

5.3 Conclusiones 48

6. DESPLIEGUE DE LA VERSIÓN 2 (CAP. 11, ACTUALIZADO) 49

6.1 Manual de despliegue, versión 2 49

6.2 Evidencia de pruebas de despliegue 51

6.3 El sistema funcionando en AWS 55

7. HISTORIAL DE COMMITS (ANEXO H) 65

8. VIDEO DE LAS PRUEBAS AUTOMATIZADAS 71
9. TECNOLOGÍAS UTILIZADAS EN CADA PUNTO 72
10. CONCLUSIONES 73

INTRODUCCIÓN

Ayni Bank es una banca 100 % digital para personas naturales en el Perú: cuenta de ahorros sin comisión de mantenimiento, apertura con el DNI y una selfie, transferencias con segundo factor, transferencias a otros bancos por CCI y tarjeta de débito virtual. Este tercer avance responde a una pregunta concreta: **¿la versión 2 funciona, se integra con sistemas externos y es usable, y se puede demostrar con pruebas automatizadas?**

La versión **2.0.0** está publicada en Amazon Web Services (https://ayni.produccion.ejemplo) desde el 30 de octubre, desplegada por el pipeline de entrega continua. Sobre esa versión corren **1 411 ejecuciones de prueba en la CI, sin un solo fallo**, 117 casos de integración contra siete sistemas externos y una suite de usabilidad con accesibilidad WCAG 2.1 AA. Todo lo que se afirma aquí tiene su captura, su reporte o su enlace.

Cómo leer este documento

El informe sigue el orden de la guía del APF3: primero el levantamiento de las observaciones del APF2 y después cada artefacto exigido. Entre paréntesis va el capítulo que le corresponde en la estructura del informe final (anexo de la guía). La tabla indica dónde está cada artefacto de la rúbrica.

Tabla 1 · Matriz de trazabilidad entre la rúbrica del APF3 y el informe

Criterio de la rúbrica	Artefacto exigido	Dónde está
Levantamiento de observaciones	Observaciones del APF2 y cómo se levantaron	Capítulo 1
Pruebas funcionales	1) Frameworks · 2) Casos de prueba · 3) Capturas de ejecución · 4) Cobertura · 5) Métricas y cumplimiento	2.1 · 2.2 · 2.3 · 2.4 · 2.5
Pruebas de integración	1) Sistemas externos · 2) Frameworks · 3) Casos · 4) Capturas · 5) Reportes · 6) Métricas	3.1 · 3.2 · 3.3 · 3.4 · 3.5 · 3.6
Usabilidad (pruebas automatizadas)	Frameworks, casos, capturas, reportes y métricas	Capítulo 4
Usabilidad según ISO/IEC 25010	Facilidad de aprendizaje, protección contra errores, asistencia y compromiso	Capítulo 5
Validación en la nube	Manual de despliegue v2 y el sistema funcionando en AWS	Capítulo 6
Historial de commits	Fecha, autor, ramas y mensajes	Capítulo 7
Sustentación	Video de las pruebas automatizadas	Capítulo 8

Nota. Elaboración propia con base en la guía y la rúbrica del Avance de Proyecto Final 3.

Enlaces del proyecto

Todo lo descrito se puede comprobar directamente: el sistema está publicado, el código y los reportes son públicos y el tablero de Jira registra el trabajo de cada integrante.

Tabla 2 · Enlaces de acceso a los recursos del proyecto

Recurso	Enlace
Sistema en producción (AWS)	https://ayni.produccion.ejemplo
Repositorio de código	https://github.com/LOAD-13/ayni-bank
CI/CD (GitHub Actions)	https://github.com/LOAD-13/ayni-bank/actions
Release v2.0.0	https://github.com/LOAD-13/ayni-bank/releases/tag/v2.0.0
Calidad (SonarCloud)	https://sonarcloud.io/project/overview?id=LOAD-13_ayni-bank
Tablero ágil (Jira)	https://jloadenegri.atlassian.net/jira/software/projects/AYNI/boards/67
Tablero de reparto (público)	https://load-13.github.io/ayni-tablero/

Nota. Elaboración propia. Los enlaces son públicos, salvo Jira, que requiere invitación al proyecto.

1. LEVANTAMIENTO DE OBSERVACIONES DEL APF2

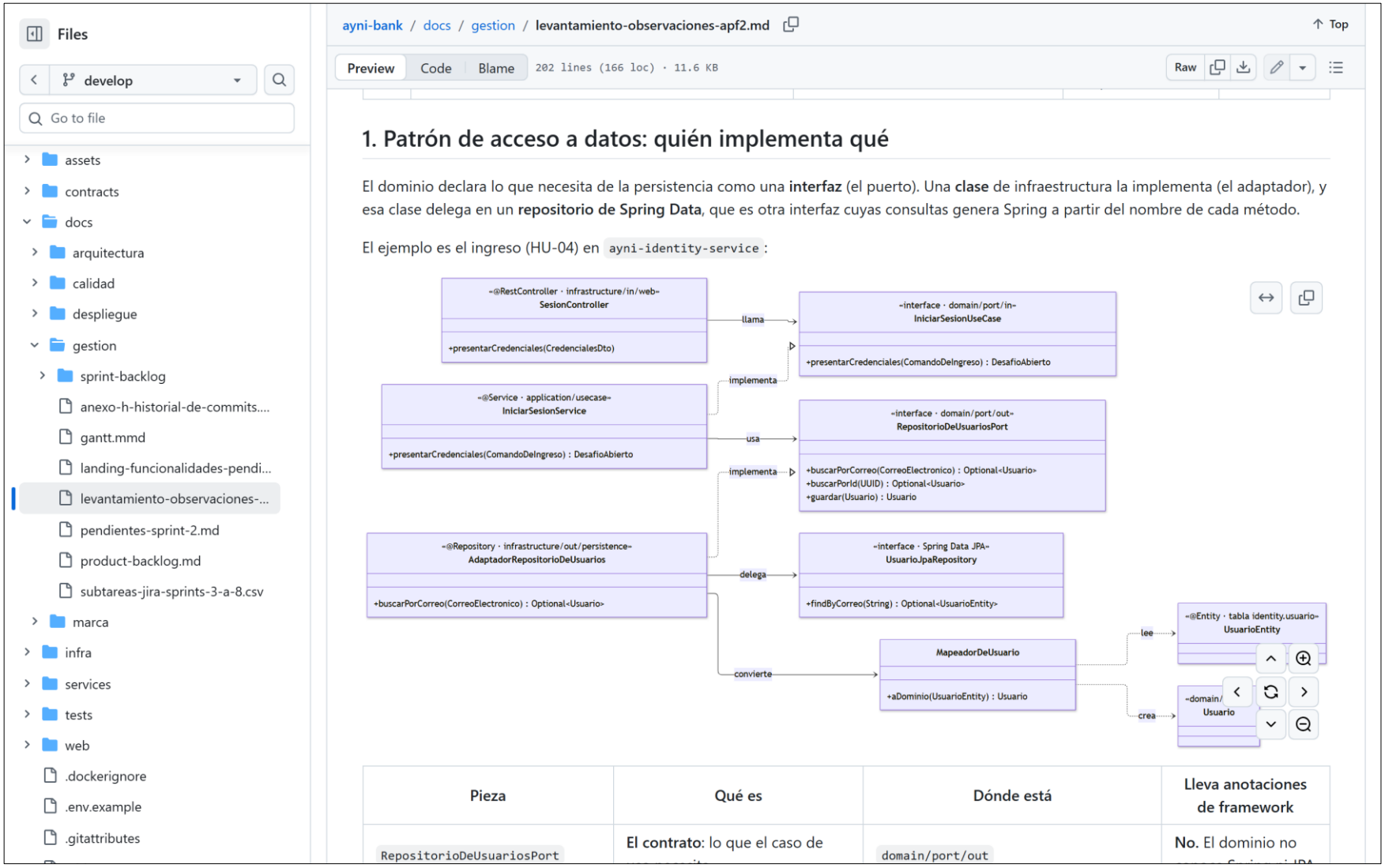
La guía pide empezar por aquí. Las siete observaciones de la sustentación del APF2 se levantaron y cada una tiene su evidencia en el documento `docs/gestion/levantamiento-observaciones-apf2.md` del repositorio.

Tabla 3 · Observaciones del APF2 y cómo se levantaron

N.º	Observación del docente	Cómo se levantó	Evidencia	Estado
1	El diagrama del patrón de acceso a datos no dejaba ver **quién implementa la interfaz** ni por qué las flechas van al dominio; «infraestructura» se leía como despliegue.	Diagrama de clases rehecho con las tres piezas separadas: puerto, adaptador y repositorio de Spring Data.	Sección 1	Levantada
2	En el diagrama de secuencia del ingreso no aparecía **dónde está la interfaz**.	Secuencia rehecha con el puerto y el adaptador como piezas distintas.	Sección 2	Levantada
3	«Argon2id **resistente a GPU**» no era la expresión adecuada: lo que importa es la resistencia a la **fuerza bruta**.	Corregido en `SECURITY.md`, explicando por qué resiste.	Sección 3	Levantada
4	**¿En qué momento se inyectan los secretos?**	Explicado con el flujo real; GitHub no guarda ninguna clave de AWS.	Sección 4	Levantada
5	**¿Quién aprueba los pull requests?** ¿Puede aprobarlos quien los creó?	Regla del repositorio y registro de aprobaciones reales.	Sección 5	Levantada
6	Mucho texto: resumir en **tablas e imágenes**.	Este documento y el informe del APF3 presentan cada punto en tablas y diagramas.	Este documento	Levantada
7	**Siempre expone el Scrum Master**: en la siguiente sustentación deben exponer los demás integrantes.	Reparto de la exposición del APF3 entre todos los integrantes.	Presentación del APF3 (T-28 · AYNI-228)	Se cierra en el APF3

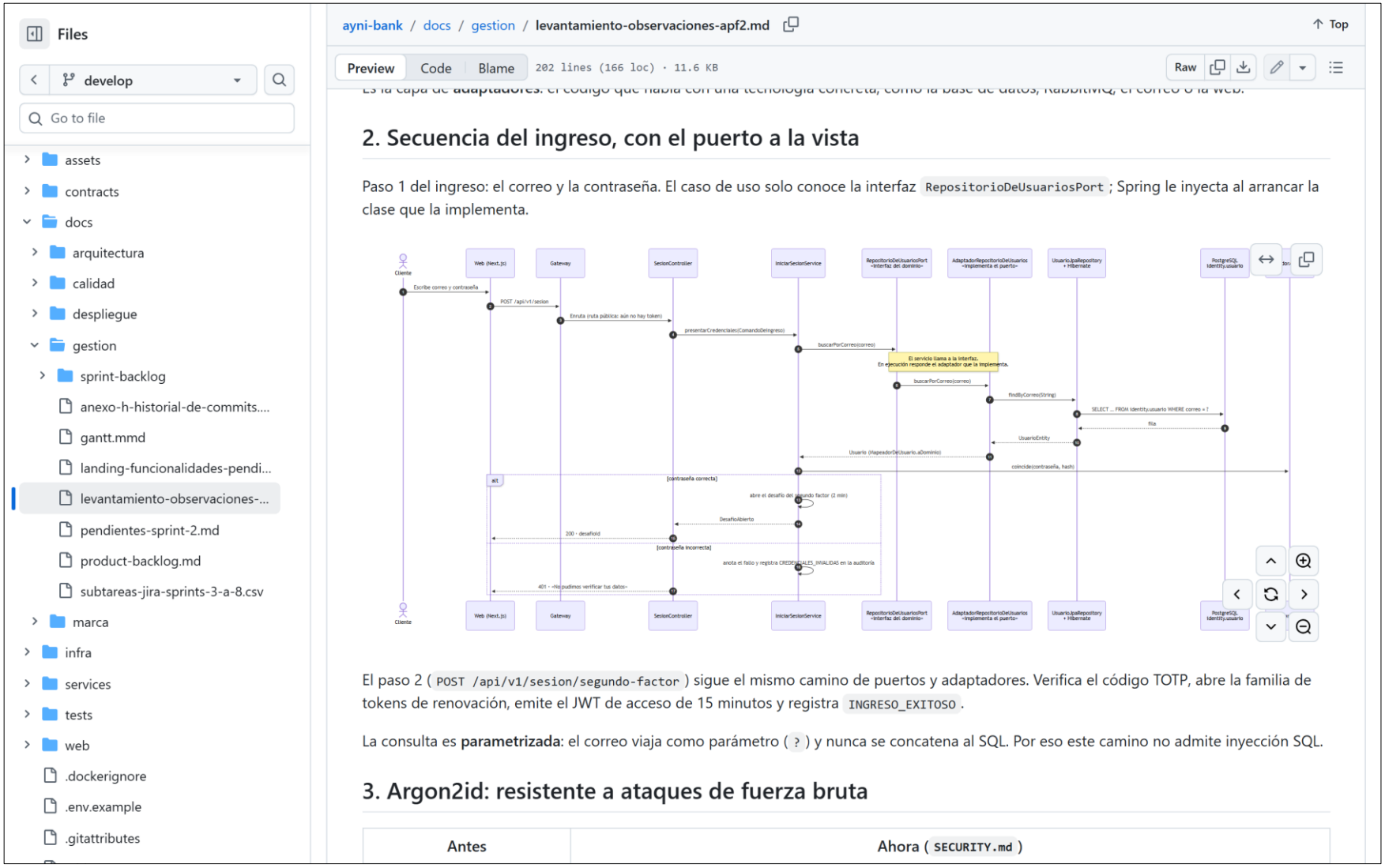
Nota. Elaboración propia a partir de la retroalimentación de la sustentación del APF2.

Figura 1 · Observación 1: el patrón de acceso a datos muestra quién implementa cada interfaz.



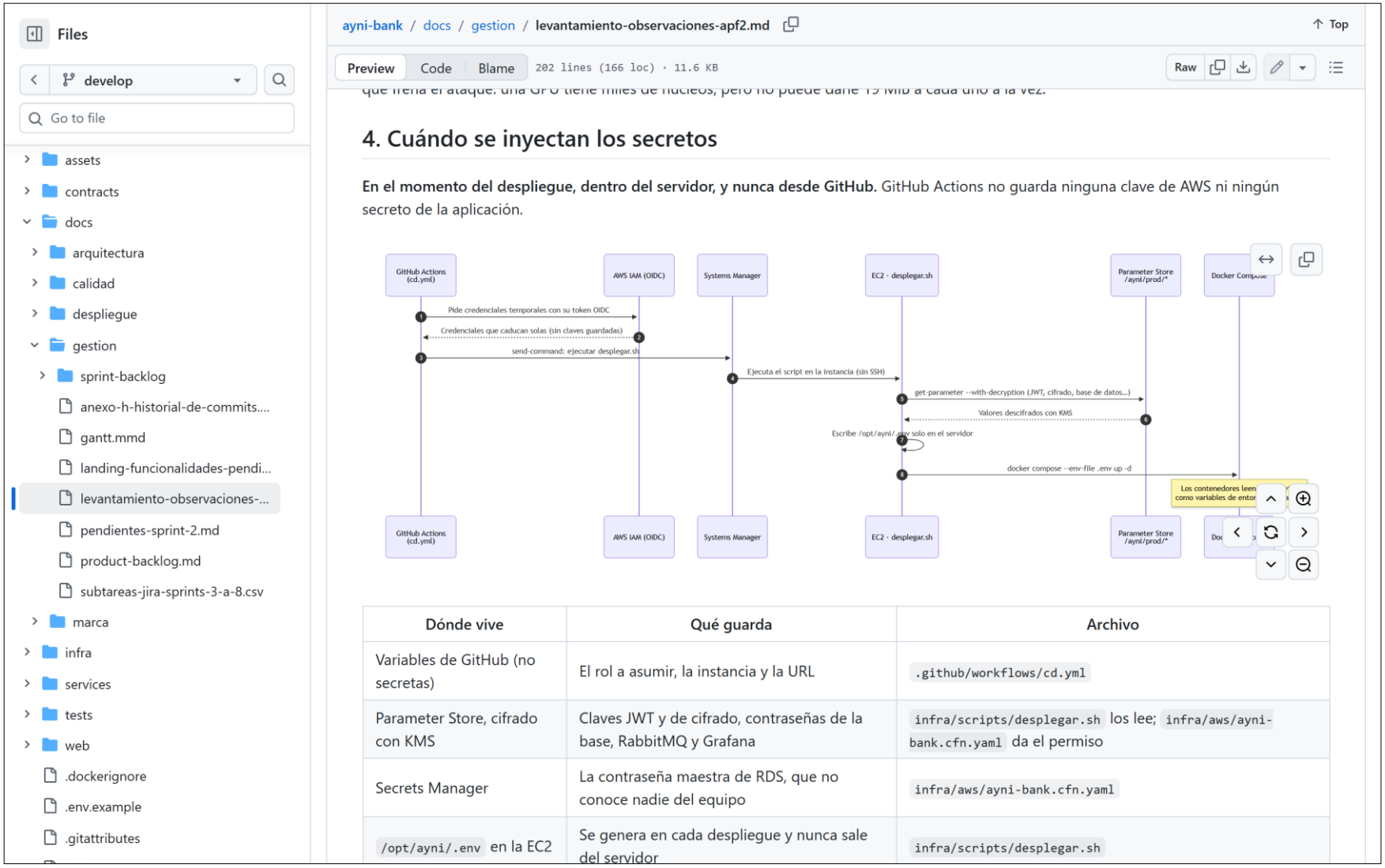
Nota. Captura del documento de levantamiento en GitHub.

Figura 2 · Observación 2: la secuencia del ingreso, con el puerto y el adaptador como piezas distintas.



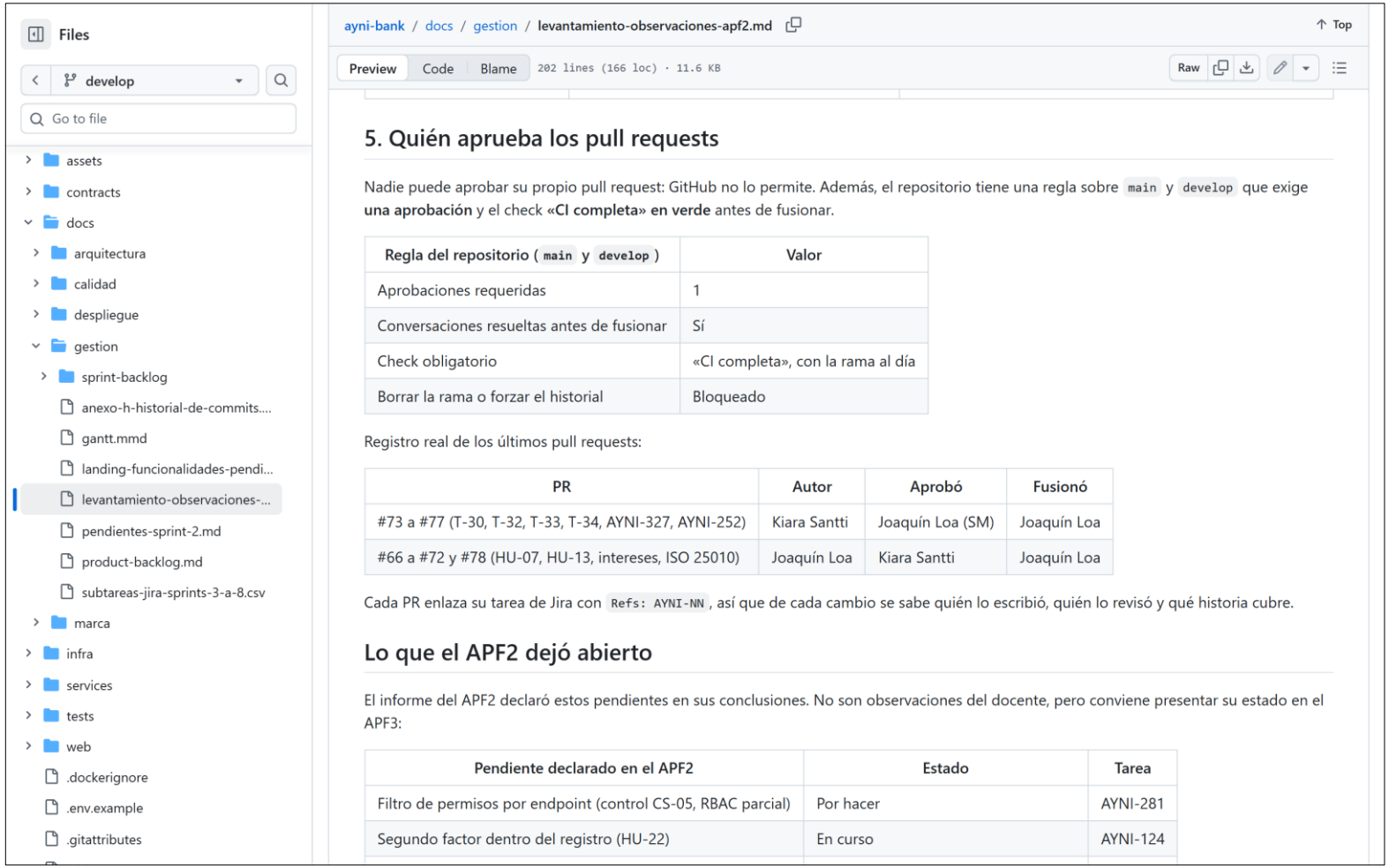
Nota. Captura del documento de levantamiento en GitHub.

Figura 3 · Observación 4: cuándo se inyectan los secretos; GitHub no guarda ninguna clave de AWS.



Nota. Captura del documento de levantamiento en GitHub.

Figura 4 · Observación 5: regla del repositorio y registro real de quién aprueba cada pull request.



Nota. Captura del documento de levantamiento en GitHub.

La observación 7 (siempre exponía el Scrum Master) se levanta en la propia sustentación: en este avance exponen los otros cinco integrantes y el Scrum Master no expone.

2. CALIDAD FUNCIONAL Y PRUEBAS AUTOMATIZADAS (CAP. 12)

La adecuación funcional de ISO/IEC 25010 se evalúa con sus tres subcaracterísticas: **completitud** (¿están todas las funciones?), **corrección** (¿dan el resultado correcto?) y **pertinencia** (¿sirven para lo que el cliente viene a hacer?). Las pruebas se organizan por niveles, del más rápido al más cercano al usuario.

Tabla 4 · Niveles de prueba del sistema

Nivel	Qué pregunta responde	Cómo
Unitaria	¿Esta pieza hace lo que dice, incluidos los errores y los límites?	Dominio sin Spring (DoD); casos de uso con dobles en memoria; componentes React aislados
Arquitectura	¿Se respeta la hexagonal?	ArchUnit: el dominio no importa Spring, JPA ni infraestructura; los puertos se llaman `*Port` y `*UseCase`
Integración	¿Las piezas reales encajan?	Testcontainers con PostgreSQL 16 y Redis reales; *slices* de Spring (`@WebMvcTest`); `TestClient` de FastAPI
Aceptación	¿Se cumplen los escenarios DADO/CUANDO/ENTONCES de la HU?	Cucumber en español contra PostgreSQL de Testcontainers, con la misma configuración que producción
Navegador (E2E)	¿El cliente puede hacerlo en la web, en escritorio y en celular?	Playwright con Chromium en dos proyectos (1280 px y 390 px)
Accesibilidad y usabilidad	¿Lo puede usar cualquiera?	axe-core (WCAG 2.1 AA) en cada ruta; suite de usabilidad ISO 25010 (T-21)
Seguridad	¿Hay secretos, CVE o fallos explotables?	Gitleaks, Trivy, SonarCloud; OWASP ZAP, Nikto, Nmap y testssl.sh contra el entorno desplegado
Humo	¿Lo que se acaba de desplegar responde?	`pruebas-humo.sh` tras cada despliegue: salud, versión, HTTPS, cabeceras, 400 y 401

Nota. Elaboración propia a partir de `docs/calidad/plan-de-pruebas.md` del repositorio.

2.1 Frameworks utilizados

Tabla 5 · Frameworks de prueba por componente

Componente	Lenguaje	Unitarias e integración	Aceptación	Cobertura
identity · core-banking · notification · gateway	Java 21 · Spring Boot 3.5	JUnit 5, Mockito, AssertJ, Spring Test, Testcontainers, ArchUnit 1.3	Cucumber 7.22	JaCoCo 0.8.12
kyc	Python 3.12 · FastAPI	pytest 8.3, httpx (`TestClient`); ruff y mypy `--strict`	Gherkin de referencia (`tests/onboarding_kyc.feature`)	pytest-cov 5.0
web	TypeScript · Next.js 15.5 · React 19	Vitest 3.2, Testing Library	Playwright 1.62 + axe-core 4.13	V8 (Vitest)

Nota. Elaboración propia a partir de `docs/calidad/plan-de-pruebas.md` del repositorio.

2.2 Listado de casos de prueba

El listado completo (**1 160 casos**, con identificador, nivel, nombre y archivo) está en `docs/calidad/casos-de-prueba.md`. Se genera desde el código con un script, de modo que no se desactualiza. Cada criterio de aceptación de las historias es además un escenario Gherkin ejecutable: **66 escenarios**, ninguno pendiente.

Tabla 6 · Casos de prueba especificados por componente y nivel

Componente	Unitaria	Arquitectura	Integración	Aceptación	Navegador	Usabilidad	Total
identity	464	5	23	31	—	—	**523**
core-banking	191	5	19	30	—	—	**245**
notification	24	5	6	5	—	—	**40**
gateway	32	—	10	—	—	—	**42**
kyc	59	—	25	—	—	—	**84**
web	165	—	—	—	38	23	**226**
Total	**935**	**15**	**83**	**66**	**38**	**23**	**1 160**

Nota. Elaboración propia a partir de `docs/calidad/casos-de-prueba.md`, generado el 30-10-2026.

Tabla 7 · Muestra del listado: escenarios de aceptación de core-banking

ID	Nivel	Caso	Archivo
CP-CORE-216	Aceptación (Gherkin)	Apertura exitosa tras aprobación del KYC	`apertura-automatica-de-cuenta.feature`
CP-CORE-217	Aceptación (Gherkin)	Intento de apertura duplicada	`apertura-automatica-de-cuenta.feature`
CP-CORE-218	Aceptación (Gherkin)	Intento de apertura duplicada · registro en la pista de auditoría	`apertura-automatica-de-cuenta.feature`
CP-CORE-219	Aceptación (Gherkin)	KYC rechazado	`apertura-automatica-de-cuenta.feature`
CP-CORE-220	Aceptación (Gherkin)	Fallo al publicar el evento	`apertura-automatica-de-cuenta.feature`
CP-CORE-221	Aceptación (Gherkin)	Congelar y descongelar la tarjeta con efecto inmediato	`control-de-tarjeta.feature`
CP-CORE-222	Aceptación (Gherkin)	Límite diario de consumo superado	`control-de-tarjeta.feature`
CP-CORE-223	Aceptación (Gherkin)	Emisión automática	`emision-de-tarjeta.feature`
CP-CORE-224	Aceptación (Gherkin)	Revelación del PAN completo	`emision-de-tarjeta.feature`
CP-CORE-225	Aceptación (Gherkin)	Sin segundo factor no se revela el PAN	`emision-de-tarjeta.feature`
CP-CORE-226	Aceptación (Gherkin)	Devengo diario correcto	`rendimiento-diario.feature`
CP-CORE-227	Aceptación (Gherkin)	Capitalización mensual	`rendimiento-diario.feature`
CP-CORE-228	Aceptación (Gherkin)	Cambio de tasa preservando el histórico	`rendimiento-diario.feature`
CP-CORE-229	Aceptación (Gherkin)	Saldo cero	`rendimiento-diario.feature`

Nota. Extracto de `docs/calidad/casos-de-prueba.md`; el listado completo tiene 1 160 filas.

2.3 Evidencia de ejecución

Las capturas son de ejecuciones reales: la suite Java completa (`mvn clean verify`), Cucumber, Playwright contra la aplicación local y la CI del pull request de la versión 2.0.0.

Figura 5 · Ejecución de Maven: pruebas por clase y BUILD SUCCESS.

Maven · Surefire (JUnit 5) · ejecución del 30-oct-2026

```
$ ./mvnw -B test -pl services/ayni-notification-service,services/ayni-gateway

[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.008 s -- in pe.ayni.bank.gateway.documentacion.DocumentacionDeLaApiTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 10.04 s -- in pe.ayni.bank.gateway.documentacion.RutasDeDocumentacionTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.056 s -- in pe.ayni.bank.gateway.limite.ConfiguracionDeLimitesTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.549 s -- in pe.ayni.bank.gateway.limite.LimitadorConRetryAfterTest
[INFO] Tests run: 9, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 10.77 s -- in pe.ayni.bank.gateway.limite.LimiteDeTasaTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.053 s -- in pe.ayni.bank.gateway.resiliencia.ManejadorDeServicioNoDisponibleTest
[INFO] Tests run: 18, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.082 s -- in pe.ayni.bank.gateway.seguridad.FiltroDeAutenticacionTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.116 s -- in pe.ayni.bank.gateway.seguridad.RutasProtegidasDeLaCuentaTest
[INFO] Results:
[INFO] Tests run: 45, Failures: 0, Errors: 0, Skipped: 0
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 3.296 s -- in pe.ayni.bank.notification.ArquitecturaHexagonalTest
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.565 s -- in pe.ayni.bank.notification.aceptacion.PruebasDeAceptacion
[INFO] Tests run: 8, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.066 s -- in pe.ayni.bank.notification.domain.model.NotificacionTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.641 s -- in pe.ayni.bank.notification.infrastructure.config.ConfiguracionDeCorreoTest
[INFO] Tests run: 10, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.861 s -- in pe.ayni.bank.notification.infrastructure.in.messaging.TraductorDeMovimientosTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.154 s -- in pe.ayni.bank.notification.infrastructure.in.programado.DespachadorProgramadoTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.062 s -- in pe.ayni.bank.notification.infrastructure.out.correo.EnviadorDeCorreoSesTest
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 12.72 s -- in pe.ayni.bank.notification.integracion.NotificacionesEnPostgresTest
[INFO] Results:
[INFO] Tests run: 40, Failures: 0, Errors: 0, Skipped: 0
[INFO] Reactor Summary for ayni-gateway 0.1.0-SNAPSHOT:
[INFO] ayni-gateway ..................................... SUCCESS [ 29.266 s]
[INFO] ayni-notification-service ........................ SUCCESS [ 24.593 s]
[INFO] BUILD SUCCESS
[INFO] Total time:  54.431 s
```

Nota. Líneas tomadas del log real; no se editaron.

Figura 6 · Escenario Gherkin de la HU-07 ejecutado por Cucumber.

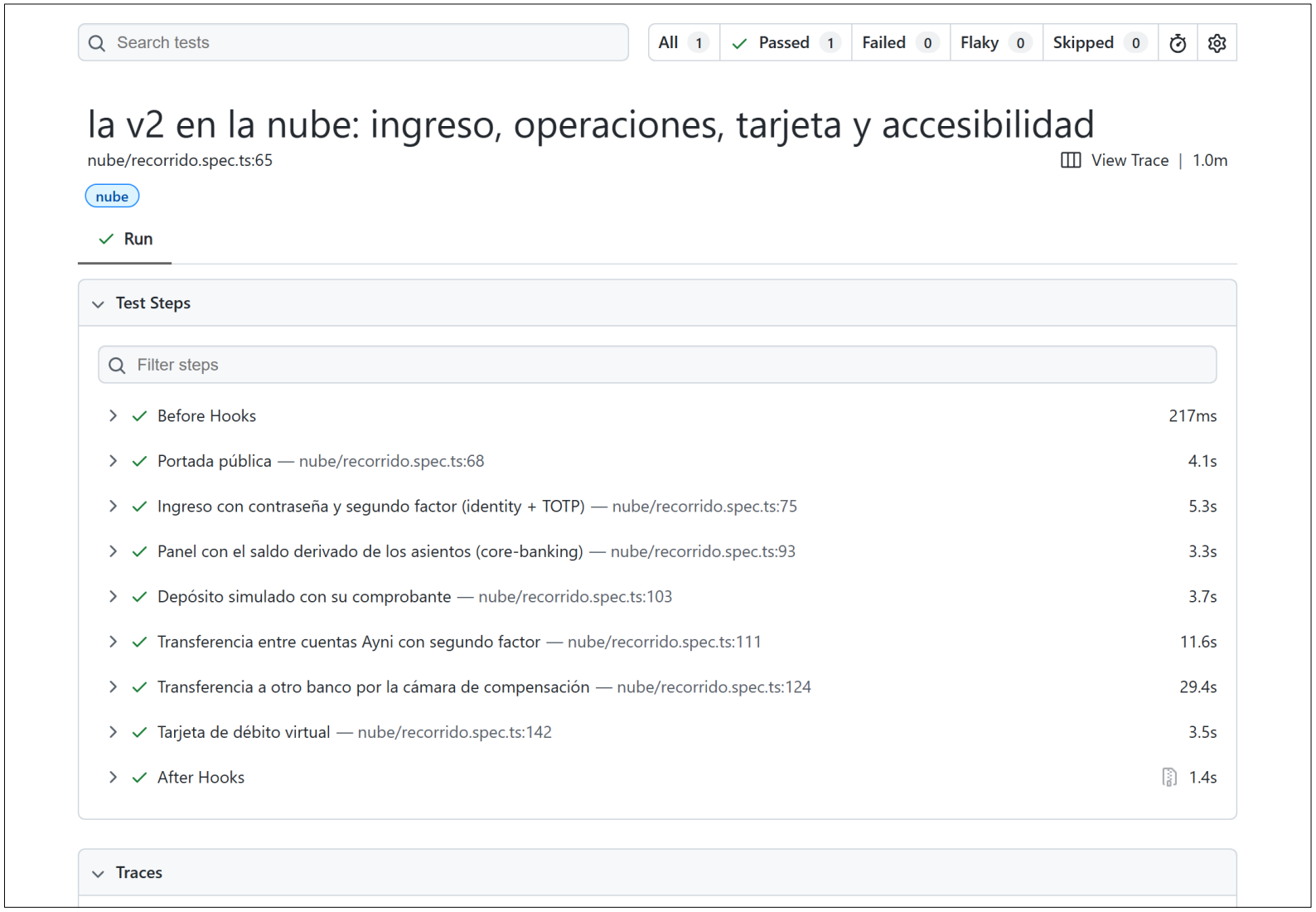
Cucumber · escenario en Gherkin ejecutado (core-banking, HU-07)

```
$ ./mvnw -B clean verify

Escenario: Transferencia exitosa con partida doble y comprobante
  Dado que Carla tiene una cuenta Ayni activa con S/ 500.00
  Y que Diego tiene una cuenta Ayni activa con S/ 100.00
  Cuando Carla transfiere S/ 150.00 a la cuenta de Diego con el concepto "Almuerzo" y confirma con su segundo factor
  Entonces el saldo de Carla queda en S/ 350.00 y el de Diego en S/ 250.00
  Y el movimiento registra un CARGO y un ABONO cuya suma algebraica es S/ 0.00
  Y se genera un comprobante válido con la cuenta destino enmascarada
```

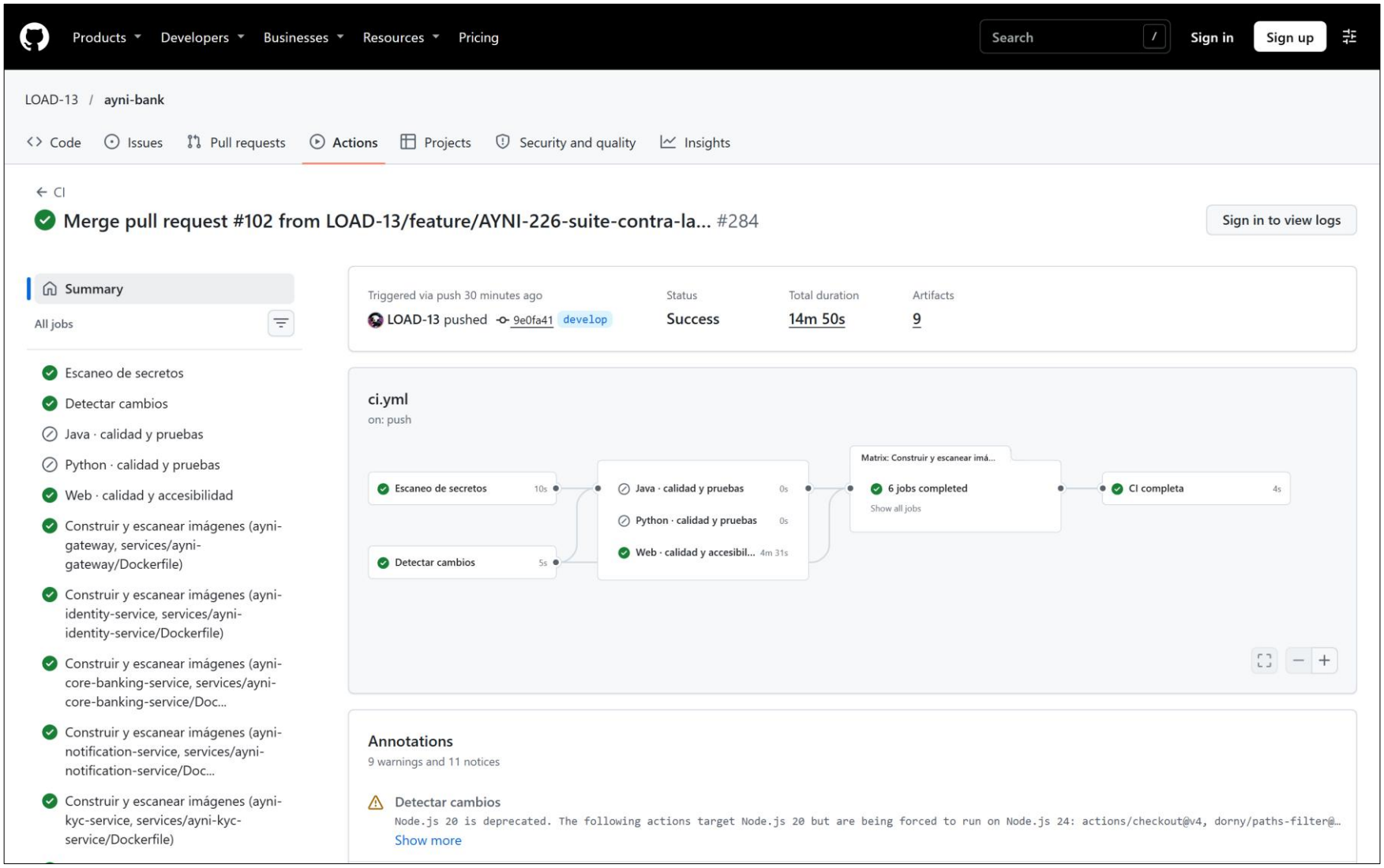
Nota. Salida real de Cucumber durante ./mvnw -B clean verify; no se editó.

Figura 7 · Recorrido de extremo a extremo con Playwright: siete pasos en verde contra los servicios reales.



Nota. Captura del reporte HTML de Playwright de la prueba nube/recorrido.spec.ts.

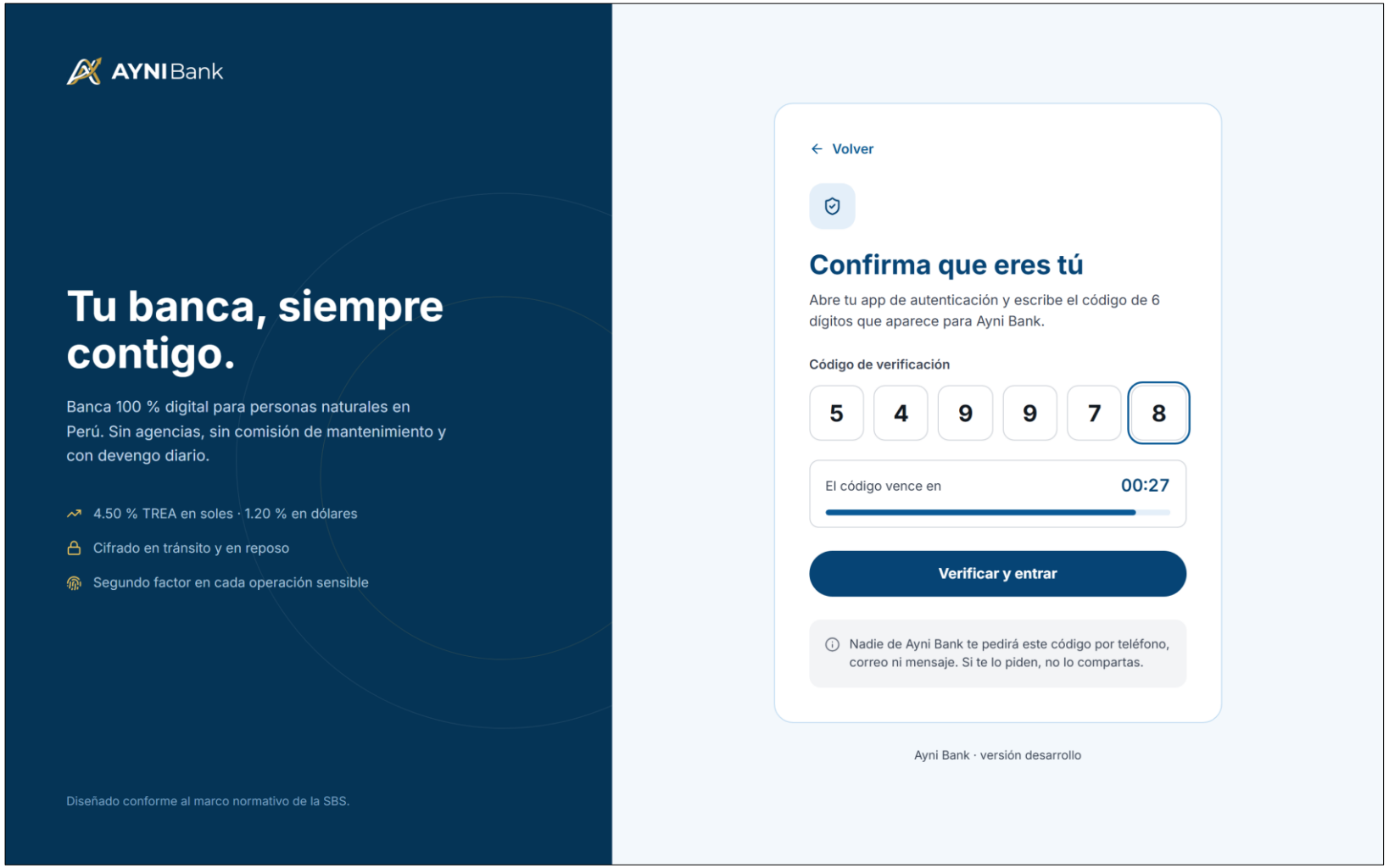
Figura 8 · La CI en GitHub Actions: «CI completa» en verde.



Nota. Captura de GitHub Actions: flujo ci.yml sobre la rama develop.

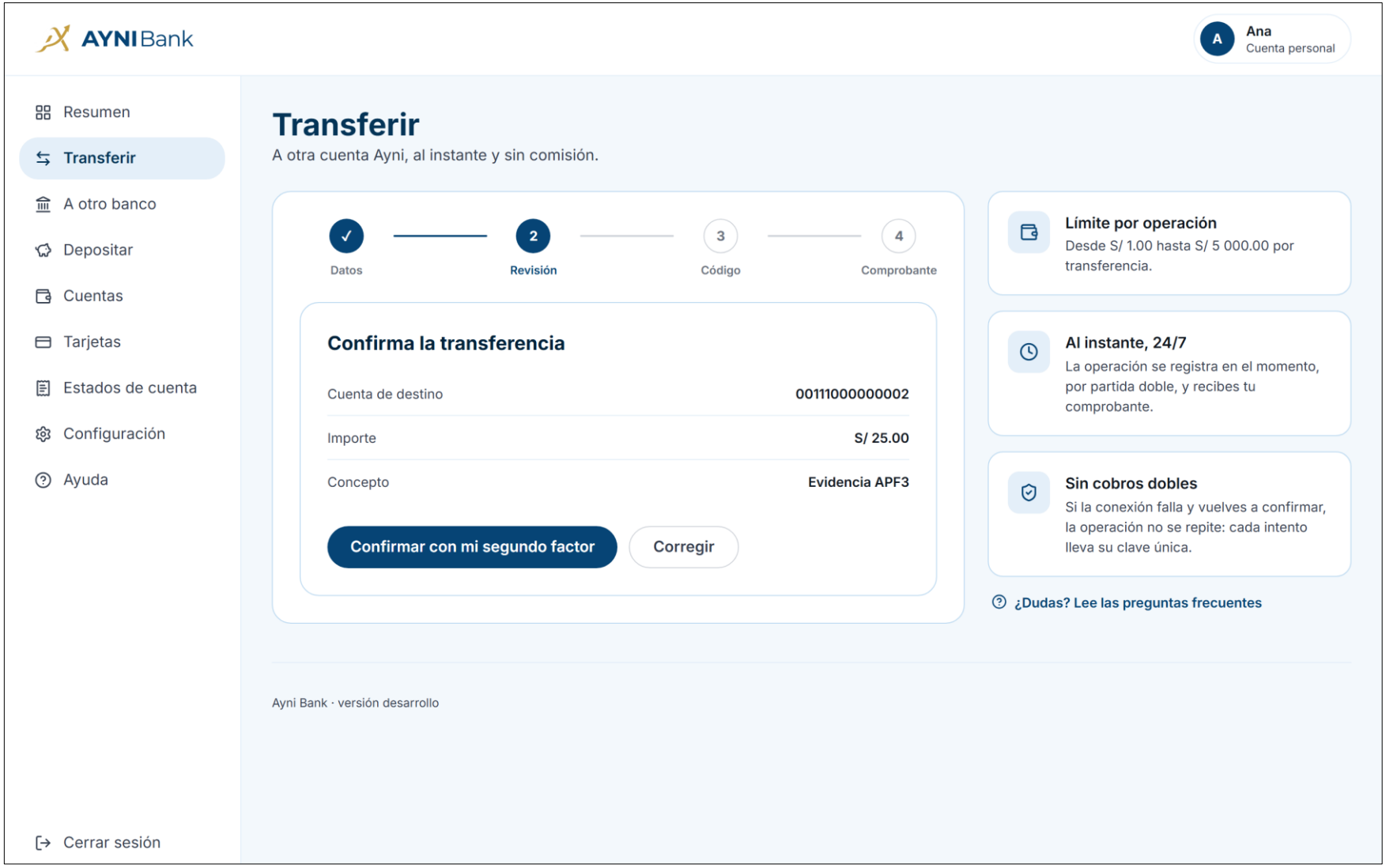
Lo que esas pruebas comprueban se ve en la propia aplicación: el ingreso con segundo factor, la transferencia en sus cuatro pasos y el comprobante.

Figura 9 · Ingreso con el código de la aplicación autenticadora (HU-04).



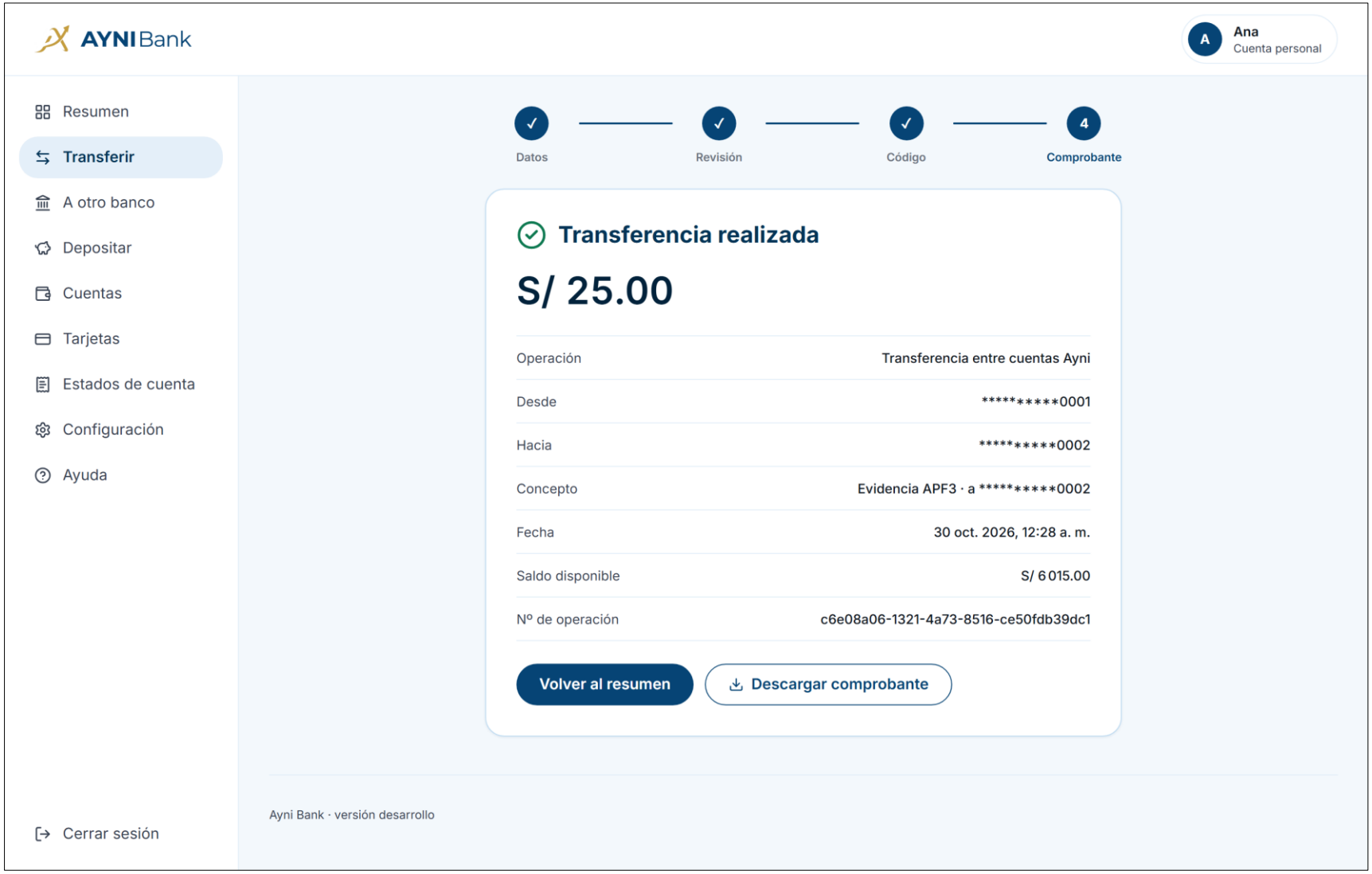
Nota. Captura de pantalla de la aplicación web de Ayni Bank.

Figura 10 · Transferencia: revisión antes de confirmar (HU-07).



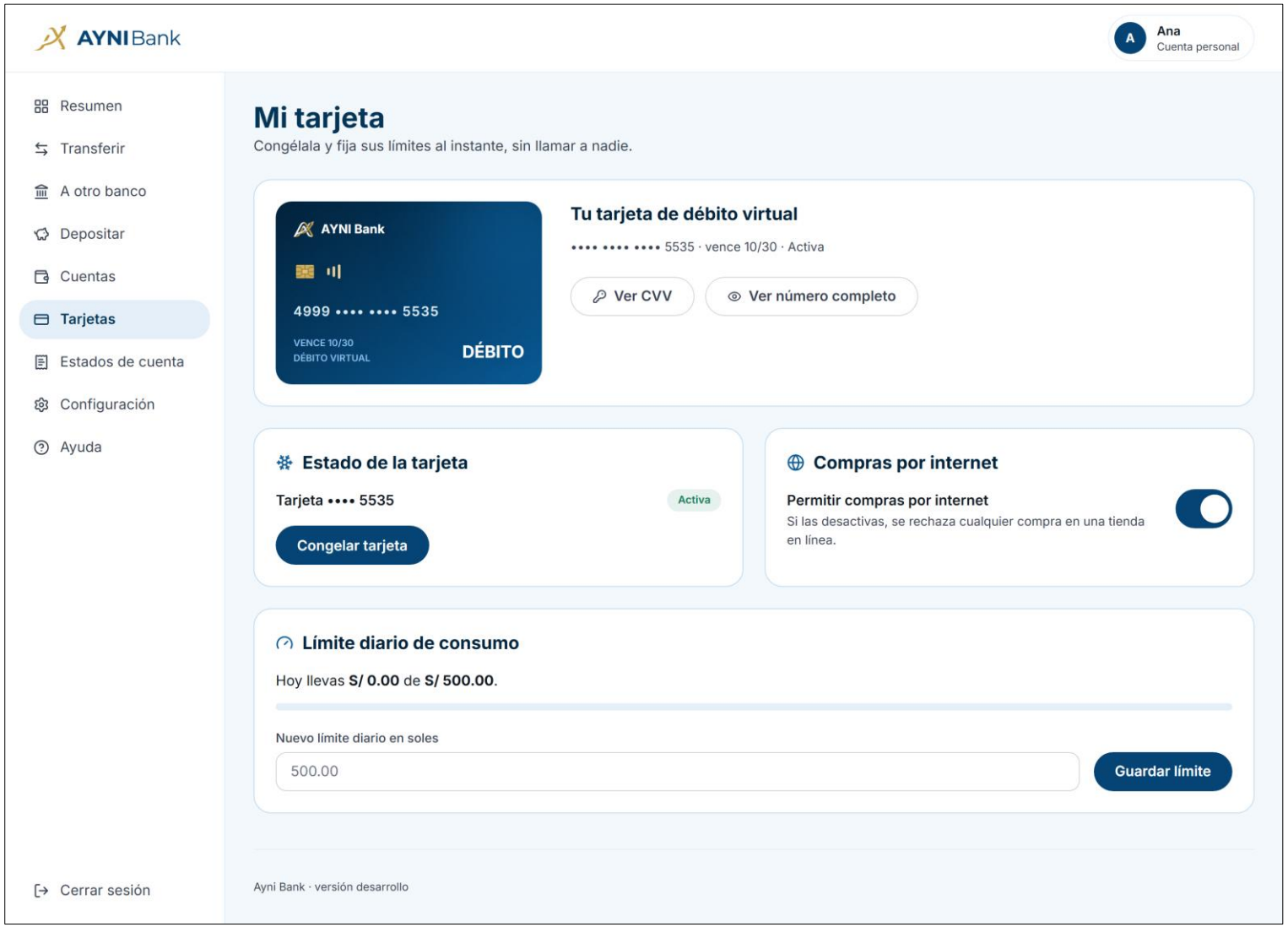
Nota. Captura de pantalla de la aplicación web de Ayni Bank.

Figura 11 · Transferencia: comprobante tras confirmar con el segundo factor.



Nota. Captura de pantalla de la aplicación web de Ayni Bank.

Figura 12 · Tarjeta de débito virtual con el PAN enmascarado (HU-09 y HU-10).



Nota. Captura de pantalla de la aplicación web de Ayni Bank.

2.4 Reportes de cobertura

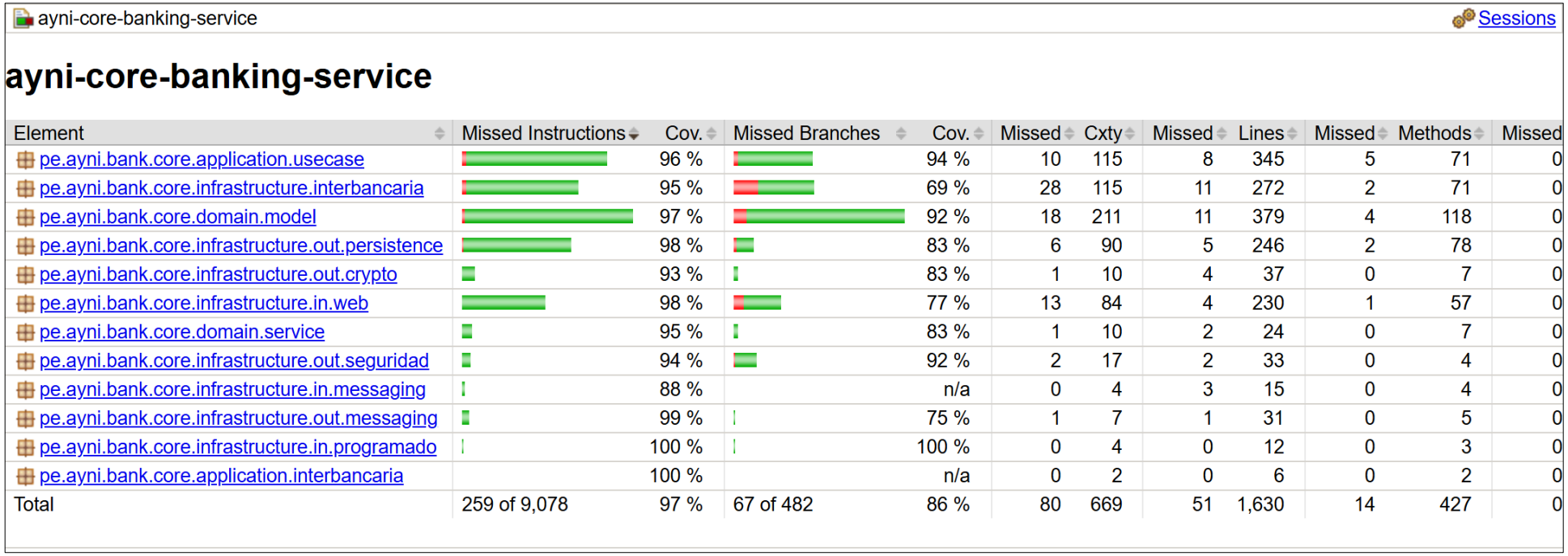
Tabla 8 · Cobertura de código por componente

Componente	Líneas	Ramas	Fuente
identity	94,4 %	83,1 %	JaCoCo
core-banking	96,9 %	86,1 %	JaCoCo
notification	99,0 %	100 %	JaCoCo
gateway	97,5 %	87,5 %	JaCoCo
kyc	94,8 %	—	coverage.py
web · total	57,9 %	86,4 %	V8 (Vitest)

Nota. Elaboración propia a partir de JaCoCo, coverage.py y Vitest (V8) sobre la versión 2.0.0.

La cobertura web total es menor porque las páginas y la portada se prueban con Playwright, que no suma a la cobertura de Vitest. La puerta de calidad de SonarCloud exige el 80 % sobre el **código nuevo** de cada pull request; la release 2.0.0 la aprobó con **94,1 %**.

Figura 13 · Reporte JaCoCo de core-banking.



ayni-core-banking-service

Sessions

ayni-core-banking-service

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed
pe.ayni.bank.core.application.usecase		96 %		94 %	10	115	8	345	5	71	0
pe.ayni.bank.core.infrastructure.interbancaria		95 %		69 %	28	115	11	272	2	71	0
pe.ayni.bank.core.domain.model		97 %		92 %	18	211	11	379	4	118	0
pe.ayni.bank.core.infrastructure.out.persistence		98 %		83 %	6	90	5	246	2	78	0
pe.ayni.bank.core.infrastructure.out.crypto		93 %		83 %	1	10	4	37	0	7	0
pe.ayni.bank.core.infrastructure.in.web		98 %		77 %	13	84	4	230	1	57	0
pe.ayni.bank.core.domain.service		95 %		83 %	1	10	2	24	0	7	0
pe.ayni.bank.core.infrastructure.out.seguridad		94 %		92 %	2	17	2	33	0	4	0
pe.ayni.bank.core.infrastructure.in.messaging		88 %		n/a	0	4	3	15	0	4	0
pe.ayni.bank.core.infrastructure.out.messaging		99 %		75 %	1	7	1	31	0	5	0
pe.ayni.bank.core.infrastructure.in.programado		100 %		100 %	0	4	0	12	0	3	0
pe.ayni.bank.core.application.interbancaria		100 %		n/a	0	2	0	6	0	2	0
Total	259 of 9,078	97 %	67 of 482	86 %	80	669	51	1,630	14	427	0

Nota. Captura del reporte HTML de JaCoCo.

Figura 14 · Reporte JaCoCo de identity.

ayni-identity-service

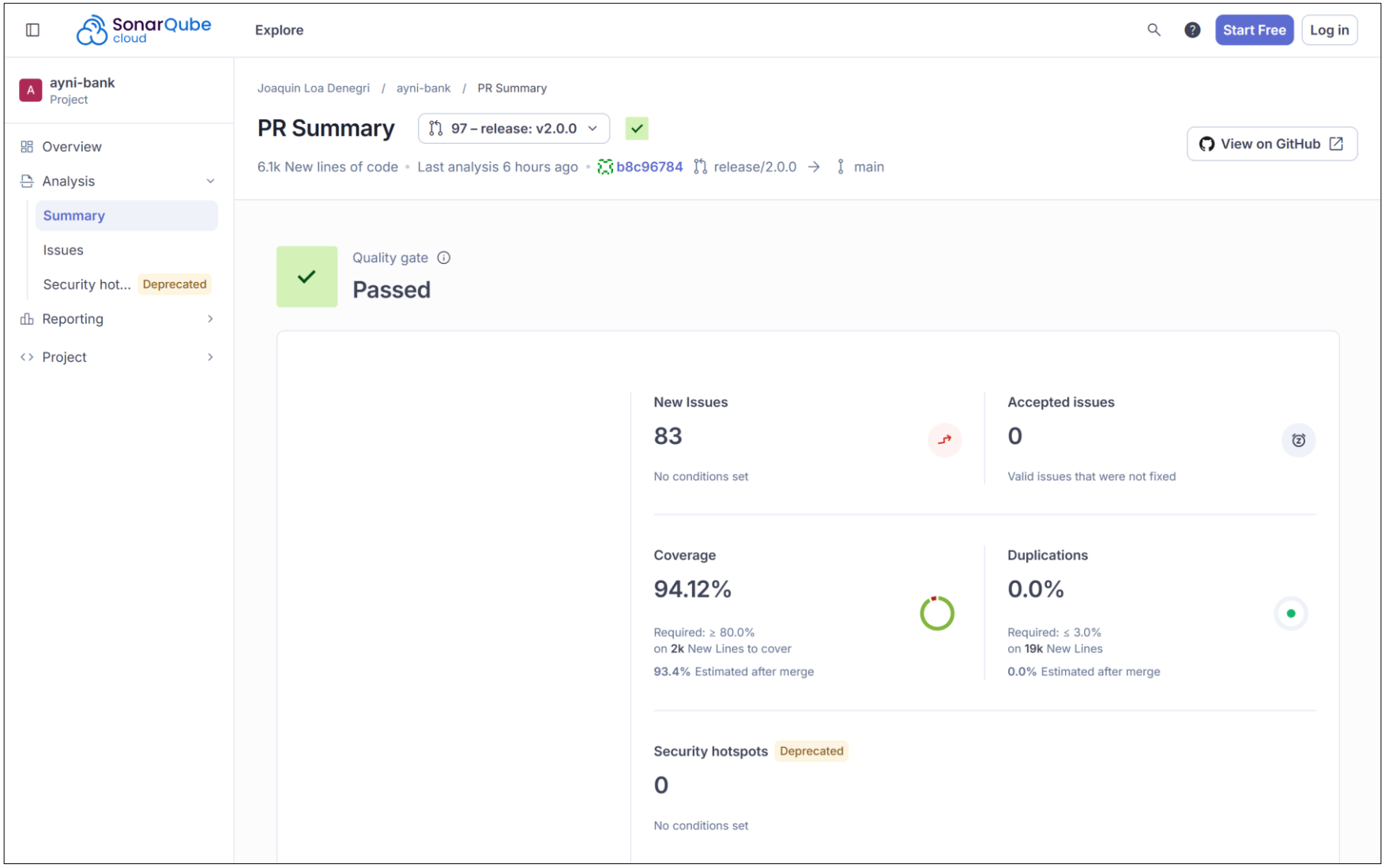
Sessions

ayni-identity-service

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Method
pe.ayni.bank.identity.domain.model		96 %		81 %	56	414	29	671	9	28
pe.ayni.bank.identity.infrastructure.out.persistence		88 %		85 %	14	95	27	254	13	8
pe.ayni.bank.identity.infrastructure.out.messaging		57 %		42 %	21	41	30	86	14	3
pe.ayni.bank.identity.application.usecase		97 %		87 %	34	237	14	755	5	11
pe.ayni.bank.identity.infrastructure.in.web		93 %		78 %	16	104	18	281	6	7
pe.ayni.bank.identity.infrastructure.out.crypto		93 %		92 %	2	31	11	127	0	1
pe.ayni.bank.identity.infrastructure.out.storage		90 %		50 %	4	12	3	45	0	
pe.ayni.bank.identity.infrastructure.out.client.kyc		97 %		75 %	3	15	2	36	0	
pe.ayni.bank.identity.infrastructure.out.correo		99 %		80 %	2	32	1	94	0	2
pe.ayni.bank.identity.infrastructure.out.persistence.entity		100 %		n/a	0	19	0	29	0	1
pe.ayni.bank.identity.domain.service		100 %		100 %	0	13	0	18	0	
pe.ayni.bank.identity.infrastructure.out.persistence.adapter		100 %		n/a	0	4	0	8	0	
Total	616 of 11,182	94 %	111 of 657	83 %	152	1,017	135	2,404	47	68

Nota. Captura del reporte HTML de JaCoCo.

Figura 15 · Puerta de calidad de SonarCloud de la release 2.0.0: aprobada.



Nota. Captura de SonarQube Cloud: resumen del pull request de la release 2.0.0.

2.5 Métricas, nivel de cumplimiento y observaciones

Tabla 9 · Resultado de la ejecución de referencia (CI de la release 2.0.0)

Suite	Ejecutadas	Pasan	Fallan	Omitidas
Java · identity	596	596	0	0
Java · core-banking	255	255	0	0
Java · notification	40	40	0	0
Java · gateway	45	45	0	0
Python · kyc (pytest; sin cambios desde la ejecución del 27-oct)	92	92	0	0
Web · unitarias (Vitest, 28 archivos)	167	167	0	0
Web · navegador, accesibilidad y usabilidad (Playwright)	216	215	0	1
Total	**1 411**	**1 410**	**0**	**1**

Nota. Elaboración propia a partir de la ejecución 40967550502 de GitHub Actions.

Tabla 10 · Adecuación funcional según ISO/IEC 25010

Subcaracterística	Medida principal	X	Nivel
Completitud funcional	Criterios de aceptación de las HU entregadas	1,00	**Alto**
Completitud funcional	HU comprometidas en sprints cerrados	1,00	**Alto**
Corrección funcional	Pruebas ejecutadas sobre la 2.0.0	1,00	**Alto**
Pertinencia funcional	Objetivos de uso O1–O5	1,00	**Alto**
Pertinencia funcional	Los seis objetivos	0,83	**Alto**

Nota. Elaboración propia a partir de `docs/calidad/reporte-calidad-funcional.md` del repositorio.

Observaciones. La omisión de Playwright es deliberada: el tamaño táctil mínimo (WCAG 2.5.8) solo se mide en el celular. Lo único funcional pendiente es la cuenta en dólares (HU-12), planificada para el Sprint 6.

3. INTEROPERABILIDAD Y PRUEBAS DE INTEGRACIÓN (CAP. 13)

La compatibilidad de ISO/IEC 25010 tiene dos caras: **interoperabilidad** (intercambiar información y usarla) y **coexistencia** (compartir recursos sin perjudicarse). Ayni Bank se comunica con siete sistemas externos y cada frontera tiene pruebas automatizadas.

3.1 Sistemas externos a integrar

Tabla 11 · Sistemas y servicios externos

#	Sistema	Tipo	Protocolo y formato	Quién lo usa	Para qué
E1	**Cámara de compensación interbancaria**	API REST de terceros (simulada en el entorno académico)	HTTP/JSON, contrato `contracts/camara-compensacion.openapi.yaml` (`POST /transferencias`, `GET /transferencias/{id}`)	core-banking	Enviar transferencias a otros bancos por CCI y conocer su resultado (HU-11)
E2	**AWS SES**	Servicio de correo en la nube	API de AWS SDK v2 (HTTPS)	identity, notification	Códigos de verificación y avisos de cada operación (HU-13, HU-22)
E3	**Amazon S3** (MinIO en local)	Almacenamiento de objetos	Protocolo S3: formulario POST firmado (SigV4) y API de objetos	navegador, identity, kyc	Fotos del DNI y la selfie del KYC (HU-02, HU-03)
E4	**Amazon RDS PostgreSQL** (PostgreSQL 16 en local)	Base de datos gestionada	JDBC / SQL, migraciones Flyway	identity, core-banking, notification	Persistencia con un schema por servicio
E5	**RabbitMQ**	Broker de mensajería	AMQP 0-9-1, eventos JSON del outbox	identity, core-banking, notification	Eventos de dominio entre servicios (apertura de cuenta, movimientos, rechazos)
E6	**Microservicios de Ayni entre sí**	APIs REST internas	HTTP/JSON con contratos OpenAPI; JWT validado en el gateway	gateway, identity, kyc, core-banking	Enrutado, autenticación y verificación de identidad
E7	**Redis**	Almacén clave-valor	RESP	gateway	Límite de tasa por IP y por titular (ADR-0035)

Nota. Elaboración propia a partir de `docs/calidad/reporte-interoperabilidad.md` del repositorio.

3.2 Frameworks utilizados

Tabla 12 · Frameworks y herramientas de la suite de integración

Herramienta	Uso en la suite
JUnit 5 + AssertJ	Base de todas las pruebas Java
Testcontainers	PostgreSQL, Redis y **MinIO reales** en contenedores durante la prueba
MockRestServiceServer (Spring)	Frontera HTTP de la cámara: comprueba la petición que sale y responde según el contrato
Cucumber (Gherkin)	Escenarios de aceptación que cruzan servicios con PostgreSQL real; el envío a RabbitMQ se verifica en su frontera (`RabbitTemplate`)
WebTestClient (Spring WebFlux)	Gateway real contra servicios simulados y Redis real
pytest	Adaptadores de kyc (almacenamiento y visión)
Playwright (`request`)	Suite de API contra la pila desplegada, en local y en la nube (capítulo 11)

Nota. Elaboración propia a partir de `docs/calidad/reporte-interoperabilidad.md` del repositorio.

3.3 Lista de casos de prueba

Tabla 13 · Casos de prueba de interoperabilidad

ID	Sistema	Qué valida	Prueba automatizada
INT-01	E1 Cámara	El simulador decide por la entidad del CCI: 097 rechaza, 098 no responde, el resto acepta	`CamaraDeCompensacionTest.decidePorEntidad`
INT-02	E1 Cámara	Reenviar o consultar una instrucción devuelve la misma decisión (idempotencia)	`CamaraDeCompensacionTest.recuerdaLaDecision`
INT-03	E1 Cámara	Una instrucción inválida responde 400 o 404, como lo haría la cámara	`CamaraDeCompensacionTest.validaLaInstruccion`
INT-04	E1 Cámara	El adaptador envía la instrucción según el contrato y lee el estado	`CamaraDeCompensacionTest.adaptadorEnviaYConsulta`
INT-05	E1 Cámara	Una respuesta sin estado no se toma por buena: el resultado queda desconocido	`CamaraDeCompensacionTest.respuestaSinEstado`
INT-06	E1 + E4 + E5	7 escenarios de HU-11 de punta a punta, con PostgreSQL real: envío, rechazo, reversión por plazo, segundo factor y auditoría	`transferencia-interbancaria.feature`
INT-07	E2 SES	Se envía desde el remitente, en UTF-8, con el HTML escapado y una versión de texto	`EnviadorDeCorreoSesTest`
INT-08	E2 SES	El cliente SES tiene timeouts de conexión, por intento y totales	`ConfiguracionDeCorreoTest`
INT-09	E3 S3	**El navegador sube con el formulario firmado a un MinIO real y el servicio lee el mismo contenido (SHA-256)**	`AlmacenDeDocumentosEnMinioTest.subidaFirmadaYLectura`
INT-10	E3 S3	**La política firmada hace que el almacenamiento rechace con 403 un tipo distinto**	`AlmacenDeDocumentosEnMinioTest.politicaRechazaOtroTipo`
INT-11	E3 S3	Un documento eliminado deja de existir para el servicio	`AlmacenDeDocumentosEnMinioTest.eliminar`
INT-12	E3 S3	La política exige clave, tipo exacto y como máximo 5 MB, y se firma con el cliente público	`MinioAlmacenDeDocumentosTest`
INT-13	E5 RabbitMQ	El evento sale con el JSON del outbox, su id como `messageId`, de forma persistente; un fallo del broker no detiene los demás	`PublicadorDeOutboxTest`
INT-14	E5 RabbitMQ	identity publica `identity.titular.activado` sin dejar el correo en una traza	`RepublicadorDeTitularesTest`
INT-15	E4 + E5	El mismo evento entregado a la vez por 8 hilos produce un solo correo; los reintentos quedan registrados	`NotificacionesEnPostgresTest`
INT-16	E5 identity → core	5 escenarios: la aprobación del KYC abre la cuenta en core-banking por el evento	`apertura-automatica-de-cuenta.feature`
INT-17	E4	Transferencias concurrentes sobre PostgreSQL real no dejan saldo negativo ni asientos duplicados	`ConcurrenciaDeTransferenciasTest`
INT-18	E4	La tarjeta guarda el PAN cifrado y su huella única en PostgreSQL real	`TarjetaEnPostgresTest`
INT-19	E6 identity → kyc	El cliente generado desde el contrato de kyc llama y lee la respuesta	`ClienteKycGeneradoTest`
INT-20	E6 gateway	El JWT se valida en el gateway; la cabecera de usuario del cliente se descarta	`FiltroDeAutenticacionTest`
INT-21	E6 gateway	Toda consulta de cuentas sin sesión responde 401, con las	`RutasProtegidasDeLaCuentaTest`

ID	Sistema	Qué valida	Prueba automatizada
		rutas reales del gateway	
INT-22	E6 + E7	Límite de tasa con Redis real: 5 inicios de sesión por minuto por IP, 429 con `Retry-After`	`LimiteDeTasaTest`
INT-23	E6	Un servicio caído se traduce en 503 con `Retry-After`, no en 500	`ManejadorDeServicioNoDisponibleTest`

Nota. Elaboración propia a partir de `docs/calidad/reporte-interoperabilidad.md` del repositorio.

3.4 Evidencia de ejecución

Figura 16 · Testcontainers levanta PostgreSQL y Redis reales durante la ejecución.

Testcontainers · contenedores reales durante las pruebas

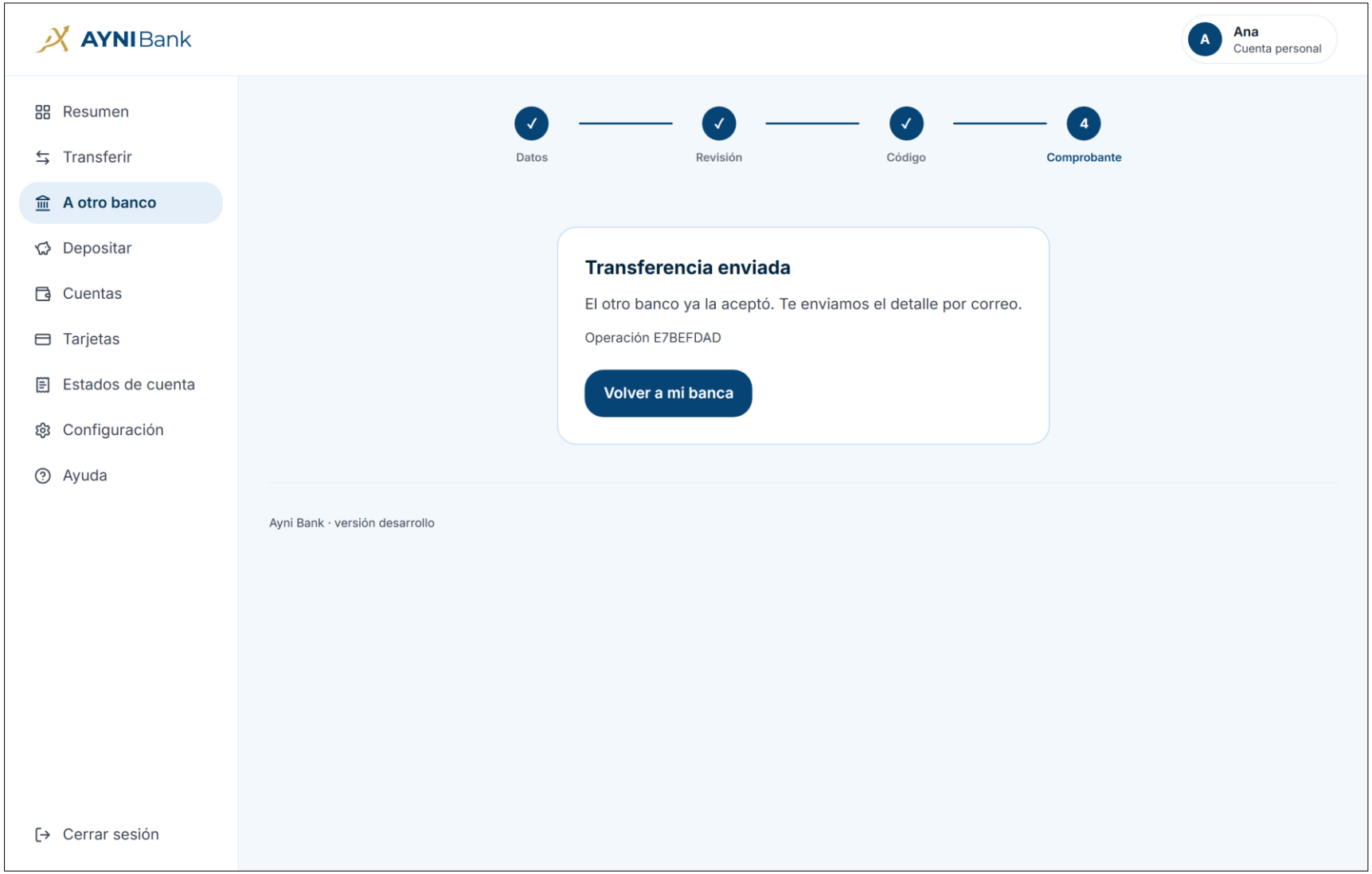
```
$ ./mvnw -B test -pl services/ayni-notification-service,services/ayni-gateway

tc.testcontainers/ryuk:0.12.0           : Creating container for image: testcontainers/ryuk:0.12.0
tc.testcontainers/ryuk:0.12.0           : Container testcontainers/ryuk:0.12.0 started in PT0.9776377S
tc.redis:7-alpine                       : Creating container for image: redis:7-alpine
tc.redis:7-alpine                       : Container redis:7-alpine started in PT0.5076501S
00:41:33.576 [main] INFO tc.testcontainers/ryuk:0.12.0 -- Creating container for image: testcontainers/ryuk:0.12.0
00:41:34.330 [main] INFO tc.testcontainers/ryuk:0.12.0 -- Container testcontainers/ryuk:0.12.0 started in PT0.7540261S
00:41:34.891 [main] INFO tc.postgres:16-alpine -- Creating container for image: postgres:16-alpine
00:41:36.959 [main] INFO tc.postgres:16-alpine -- Container postgres:16-alpine started in PT2.0678907S

[INFO] Tests run: 9, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 10.77 s -- in pe.ayni.bank.gateway.limite.LimiteDeTasaTest
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.565 s -- in pe.ayni.bank.notification.aceptacion.PruebasDeAceptacion
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 12.72 s -- in pe.ayni.bank.notification.integracion.NotificacionesEnPostgresTest
```

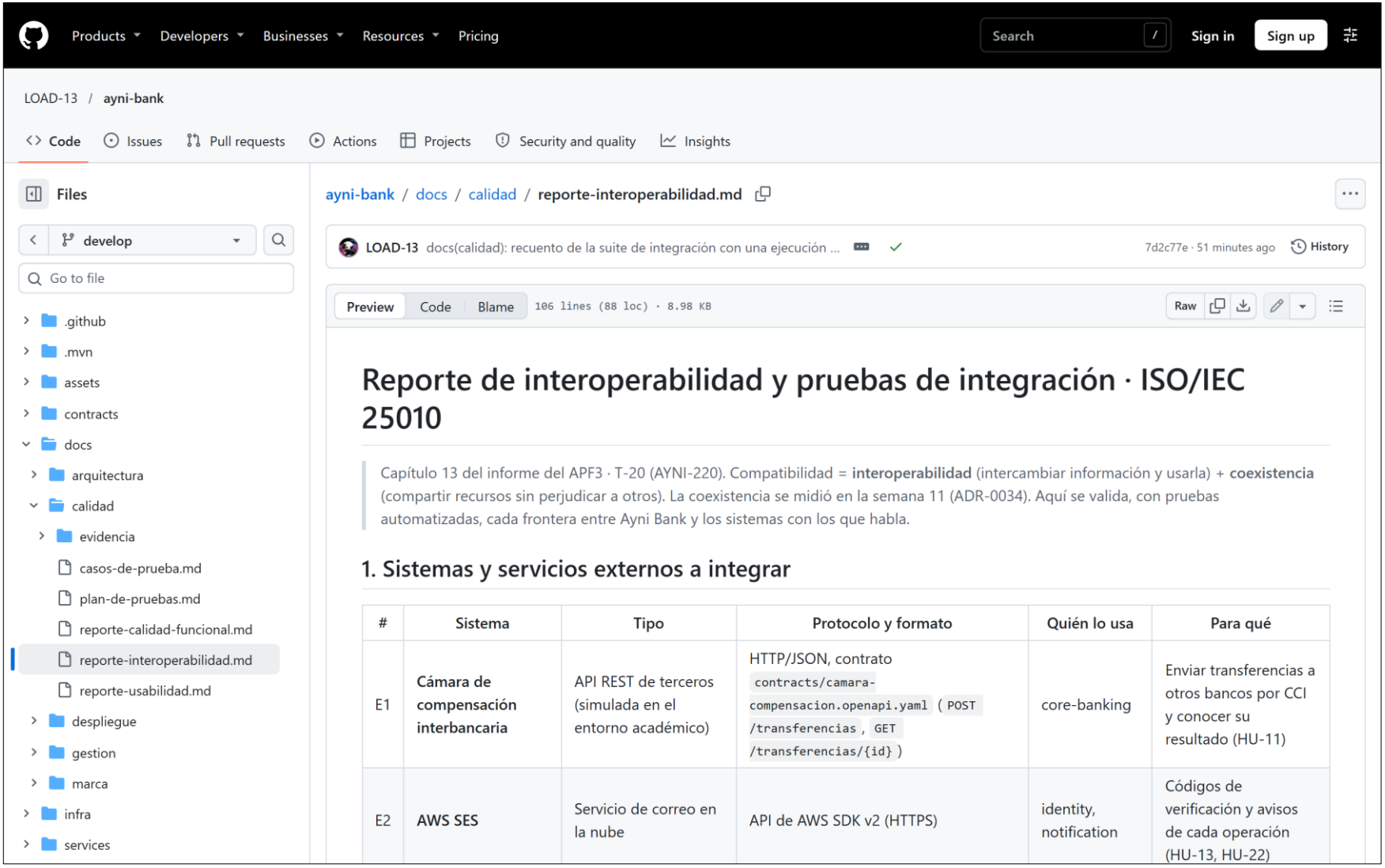
Nota. Líneas del log real de Maven.

Figura 17 · Interbancaria aceptada por la cámara de compensación, a través de su contrato OpenAPI.



Nota. Captura de pantalla de la aplicación web de Ayni Bank.

Figura 18 · El reporte de interoperabilidad en el repositorio.



Nota. Captura de docs/calidad/reporte-interoperabilidad.md en GitHub.

3.5 Reporte de resultados

Tabla 14 · Resultados de la suite de integración por sistema

Sistema y frontera	Casos	Pasan
E1 · Cámara de compensación (API REST)	9	9
E1 · Cámara de compensación (API REST, contrato OpenAPI)	5	5
E2 · AWS SES (correo)	5	5
E3 · S3 / MinIO	12	12
E3 · S3 / MinIO real (Testcontainers)	3	3
E4 · PostgreSQL real (Testcontainers)	14	14
E4 · PostgreSQL real y E5 · RabbitMQ en su frontera (Cucumber)	30	30
E5 · RabbitMQ (AMQP)	5	5
E6 · Autenticación JWT entre servicios	22	22
E6 · Microservicio kyc (API REST)	1	1
E6 · Microservicios (coexistencia)	2	2
E7 · Redis real (Testcontainers) + microservicios	9	9
Total	**117**	**117**

Nota. Elaboración propia a partir de los XML de Surefire de `mvn clean verify` del 30-10-2026.

3.6 Métricas, nivel de cumplimiento y observaciones

Tabla 15 · Métricas de la suite de integración

Métrica	Fórmula	Resultado
Cumplimiento de la suite	casos que pasan / casos ejecutados	**100 %** (117 de 117, `mvn clean verify` local del 30-oct-2026 sobre la 2.0.0)
Sistemas externos cubiertos	sistemas con al menos una prueba de su frontera / sistemas identificados	**7 de 7**
Fronteras con el protocolo real	sistemas probados contra una instancia real (Testcontainers)	3 de 7: PostgreSQL, Redis y S3/MinIO. RabbitMQ, SES y la cámara se prueban en su frontera
Tiempo de la suite	suma de los tiempos medidos	21,3 s

Nota. Elaboración propia a partir de `docs/calidad/reporte-interoperabilidad.md` del repositorio.

Observaciones. SES, la cámara y RabbitMQ se prueban en su frontera, no contra el servicio real: SES necesitaría credenciales de AWS en la CI y la cámara no existe fuera del entorno académico. El intercambio real con todos ellos se comprueba en la nube (capítulo 6). Una mejora pendiente es probar RabbitMQ también contra un broker real con Testcontainers.

4. PRUEBAS AUTOMATIZADAS DE USABILIDAD (CAP. 14)

4.1 Frameworks utilizados

Tabla 16 · Frameworks de la suite de usabilidad

Herramienta	Uso
Playwright	Navegador real (Chromium) en escritorio y en un celular emulado (Pixel 7): recorre la interfaz como un usuario
axe-core (@axe-core/playwright)	Revisa cada pantalla con las reglas de WCAG 2.1 nivel AA
Vitest + Testing Library	Componentes de la interfaz: asistente, pistas, logro, validaciones
GitHub Actions	Ejecuta la suite en cada pull request y publica el reporte HTML

Nota. Elaboración propia.

4.2 Lista de casos de prueba

Tabla 17 · Casos de la suite de usabilidad

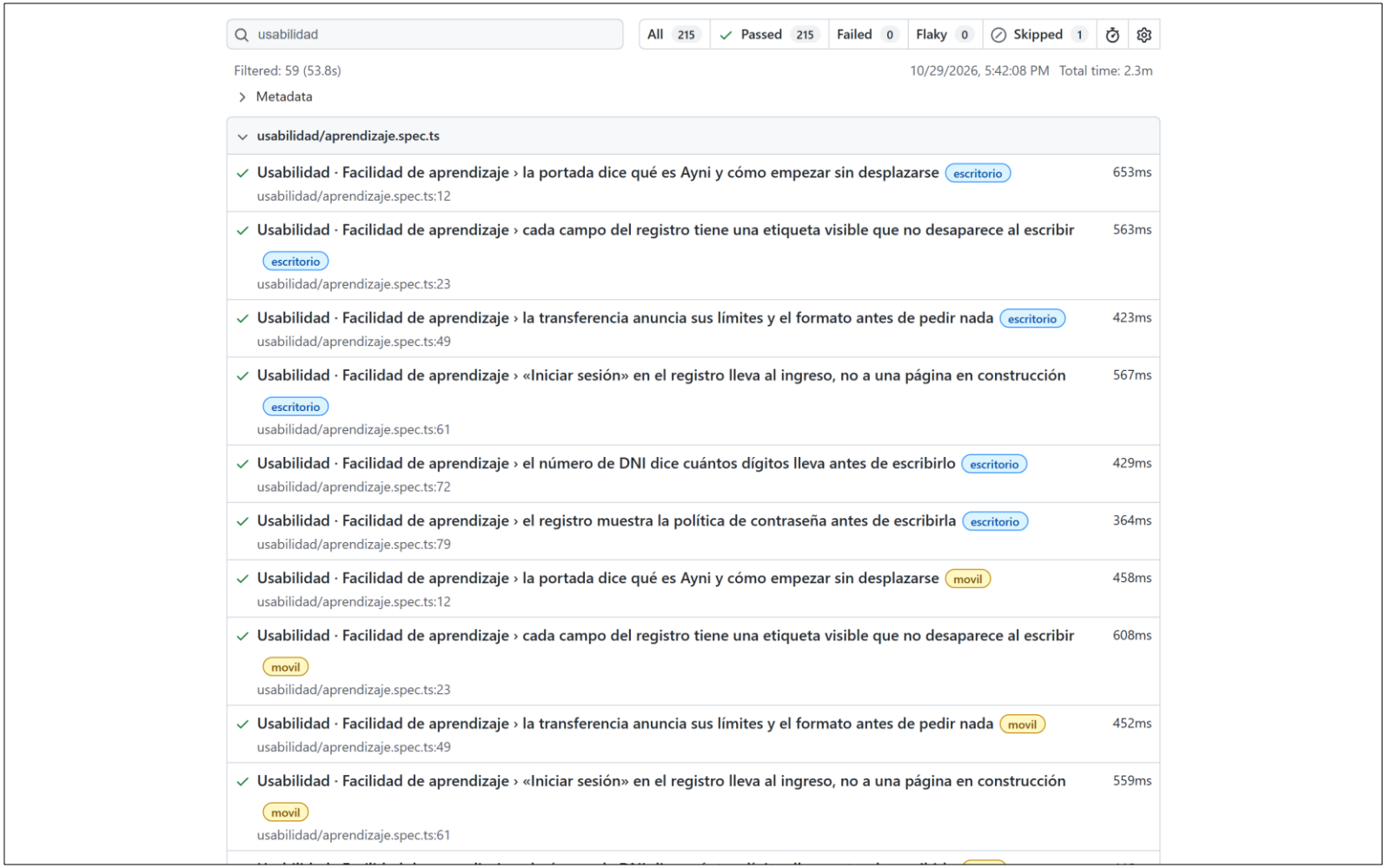
ID	Nivel	Caso	Archivo
CP-WEB-204	Usabilidad	la portada dice qué es Ayni y cómo empezar sin desplazarse	`aprendizaje.spec.ts`
CP-WEB-205	Usabilidad	cada campo del registro tiene una etiqueta visible que no desaparece al escribir	`aprendizaje.spec.ts`
CP-WEB-206	Usabilidad	la transferencia anuncia sus límites y el formato antes de pedir nada	`aprendizaje.spec.ts`
CP-WEB-207	Usabilidad	«Iniciar sesión» en el registro lleva al ingreso, no a una página en construcción	`aprendizaje.spec.ts`
CP-WEB-208	Usabilidad	el número de DNI dice cuántos dígitos lleva antes de escribirlo	`aprendizaje.spec.ts`
CP-WEB-209	Usabilidad	el registro muestra la política de contraseña antes de escribirla	`aprendizaje.spec.ts`
CP-WEB-210	Usabilidad	cada error del registro queda enlazado al campo que hay que corregir	`asistencia.spec.ts`
CP-WEB-211	Usabilidad	si la red falla al ingresar, el mensaje dice qué hacer	`asistencia.spec.ts`
CP-WEB-212	Usabilidad	si no cargan los movimientos, el panel dice qué hacer	`asistencia.spec.ts`
CP-WEB-213	Usabilidad	un error de credenciales se anuncia y deja los datos para reintentar	`asistencia.spec.ts`
CP-WEB-214	Usabilidad	guía la primera vez, es accesible y no vuelve a salir	`compromiso.spec.ts`
CP-WEB-215	Usabilidad	la pista de un acceso rápido se ve también al llegar con el tabulador	`compromiso.spec.ts`
CP-WEB-216	Usabilidad	desde una operación se llega a las preguntas frecuentes	`compromiso.spec.ts`
CP-WEB-217	Usabilidad	con «reducir movimiento» activado, nada sigue animándose	`inclusividad.spec.ts`
CP-WEB-218	Usabilidad	${ruta} no presenta violaciones de WCAG 2.1 AA con la sesión abierta *(se repite por cada valor del bucle)*	`inclusividad.spec.ts`
CP-WEB-219	Usabilidad	el ingreso se completa solo con el teclado	`operabilidad.spec.ts`
CP-WEB-220	Usabilidad	el tabulador recorre el registro en el orden en que se lee	`operabilidad.spec.ts`
CP-WEB-221	Usabilidad	${ruta}: cada parada del tabulador muestra dónde está el foco *(se repite por cada valor del bucle)*	`operabilidad.spec.ts`

ID	Nivel	Caso	Archivo
CP-WEB-222	Usabilidad	en el celular, cada control mide al menos 24 × 24 px	`operabilidad.spec.ts`
CP-WEB-223	Usabilidad	antes de mover el dinero muestra la operación para revisarla	`proteccion-contra-errores.spec.ts`
CP-WEB-224	Usabilidad	«Corregir» devuelve el formulario con lo que ya se escribió	`proteccion-contra-errores.spec.ts`
CP-WEB-225	Usabilidad	pulsar «Confirmar» dos veces seguidas pide una sola confirmación	`proteccion-contra-errores.spec.ts`
CP-WEB-226	Usabilidad	un importe no válido no avanza, marca el campo y dice cómo arreglarlo	`proteccion-contra-errores.spec.ts`

Nota. Extracto de `docs/calidad/casos-de-prueba.md`: los casos de nivel «Usabilidad».

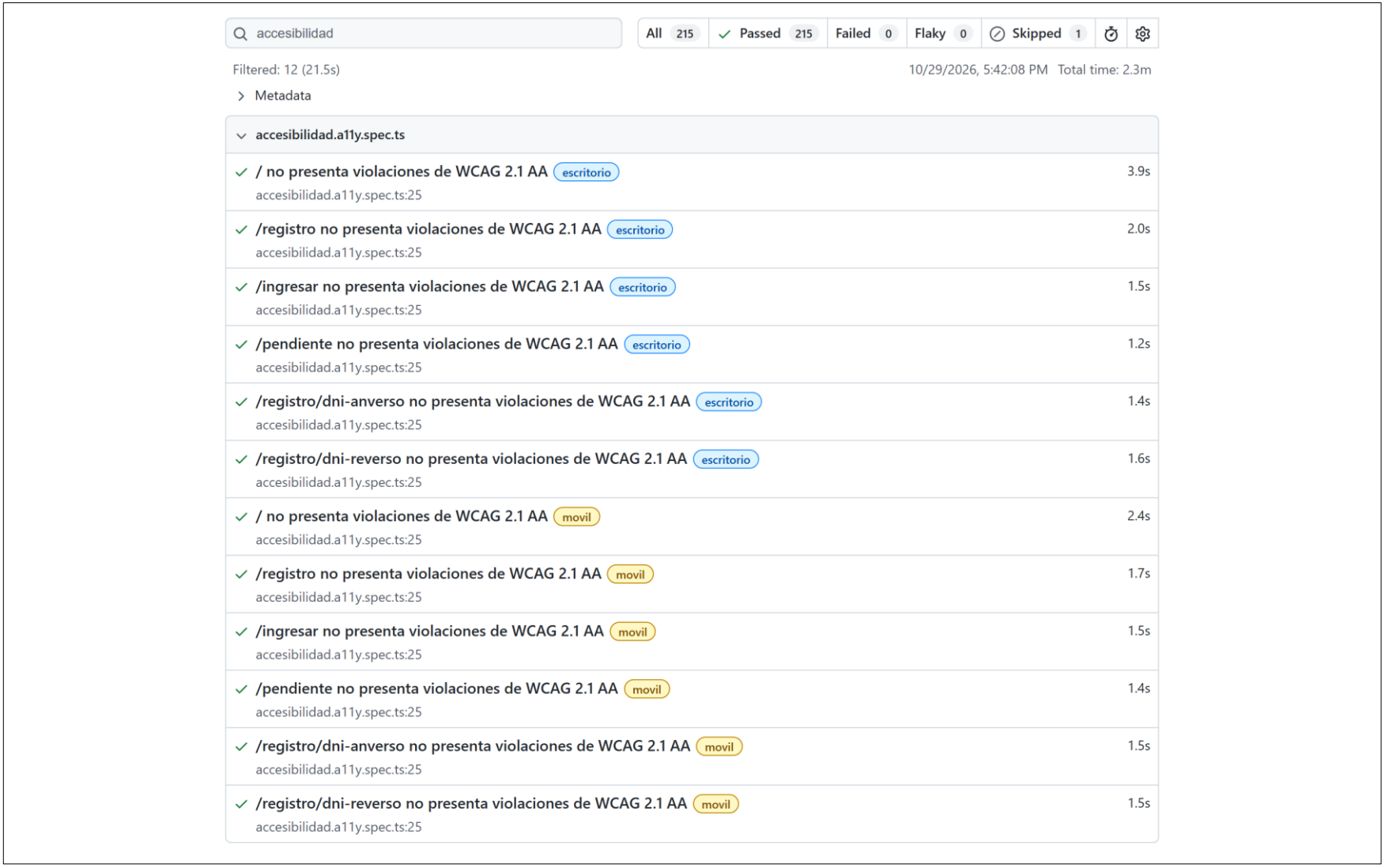
4.3 Evidencia de ejecución

Figura 19 · Reporte de Playwright: las pruebas de usabilidad en verde.



Nota. Captura del reporte HTML de Playwright, filtrado por «usabilidad».

Figura 20 · Las pantallas sin violaciones de WCAG 2.1 AA (axe-core).



Nota. Captura del reporte HTML de Playwright, filtrado por «accesibilidad».

4.4 Reportes de ejecución

Tabla 18 · Resultado de la suite de usabilidad por subcaracterística

Subcaracterística	Pruebas	Resultado del corte
Facilidad de aprendizaje	6	Todas pasan
Operabilidad	5	Todas pasan (la de tamaño táctil solo se mide en el celular)
Protección contra errores	4	Todas pasan
Asistencia al usuario	4	Todas pasan
Inclusividad	5	Todas pasan
Total	**24**	**47 ejecuciones aprobadas y 1 omitida a propósito**

Nota. Elaboración propia a partir de `docs/calidad/reporte-usabilidad.md` del repositorio.

4.5 Métricas, nivel de cumplimiento y observaciones

La suite corre en cada pull request; si una pantalla pierde una etiqueta, un mensaje de error o el contraste, el cambio no se integra. Las métricas por subcaracterística se presentan en el capítulo 5.

5. INFORME DE EVALUACIÓN DE USABILIDAD SEGÚN ISO/IEC 25010 (CAP. 15)

Se evalúa la aplicación web de la banca (registro, ingreso, panel, transferencias, tarjeta y ayuda) con las cuatro subcaracterísticas que pide la guía. Cada medida es una razón: $X = 1 - \frac{A}{B}$

Donde A son los casos que fallan el criterio y B los evaluados; 1,00 es el mejor valor.

5.1 Criterios, métricas y evidencias empleadas

Tabla 19 · Criterios y medidas por subcaracterística

Subcaracterística	Pregunta	Medida	A	B
Facilidad de aprendizaje	¿Se entiende sin manual qué pide cada pantalla?	Campos con etiqueta visible	Campos sin etiqueta	Campos de entrada
		Formatos anunciados de antemano	Campos que exigen un formato y no lo dicen hasta fallar	Campos que exigen un formato
Protección contra errores del usuario	¿La interfaz evita los errores evitables y deja corregir los demás?	Acciones sensibles protegidas	Acciones sin la protección que necesitan	Acciones que mueven dinero o cambian la seguridad
Asistencia al usuario	¿La pueden usar personas con capacidades distintas y, cuando algo falla, saben qué hacer?	Mensajes de error comprensibles	Mensajes que no dicen qué pasó **y** qué hacer	Mensajes de error que ve el cliente
		Pantallas sin barreras de accesibilidad	Pantallas con violaciones de WCAG 2.1 AA	Pantallas evaluadas con axe-core
Compromiso del usuario	¿Resulta agradable y motiva a seguir usándola?	Puntuación SUS de usuarios representativos	—	—

Nota. Elaboración propia a partir de `docs/calidad/reporte-usabilidad.md` del repositorio.

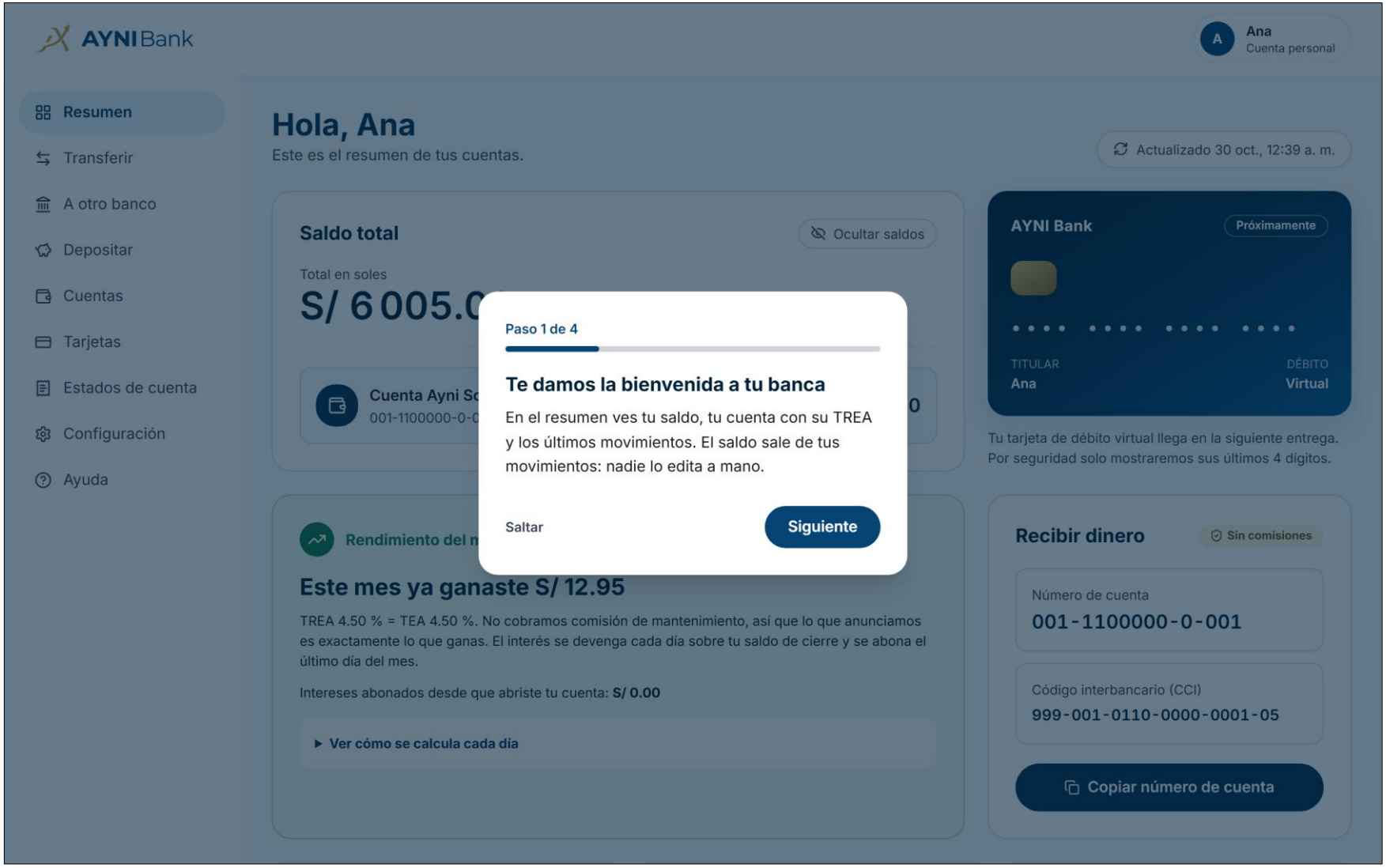
5.1.1 Facilidad de aprendizaje

Tabla 20 · Medidas de facilidad de aprendizaje

Medida	B	A antes	X antes	A ahora	X ahora	Nivel
Campos con etiqueta visible	18	0	1,00	0	**1,00**	Alto
Formatos anunciados de antemano	11	1	0,91	0	**1,00**	Alto

Nota. Elaboración propia a partir de `docs/calidad/reporte-usabilidad.md` del repositorio.

Figura 21 · Asistente de bienvenida en el primer ingreso.



Nota. Captura de pantalla de la aplicación web de Ayni Bank.

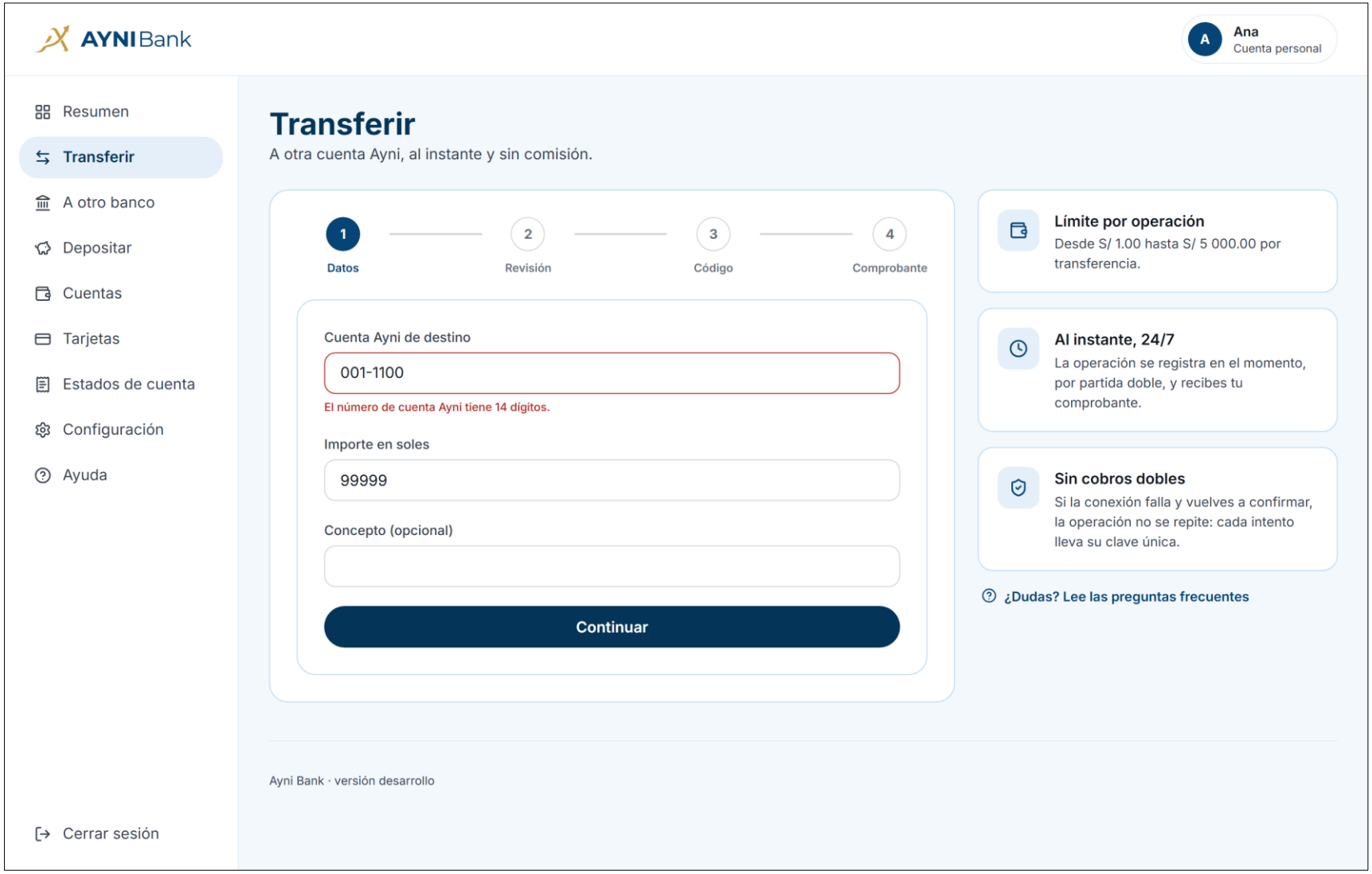
5.1.2 Protección contra errores del usuario

Tabla 21 · Medidas de protección contra errores

Medida	B	A antes	X antes	A ahora	X ahora	Nivel
Acciones sensibles protegidas	10	1	0,90	0	**1,00**	Alto

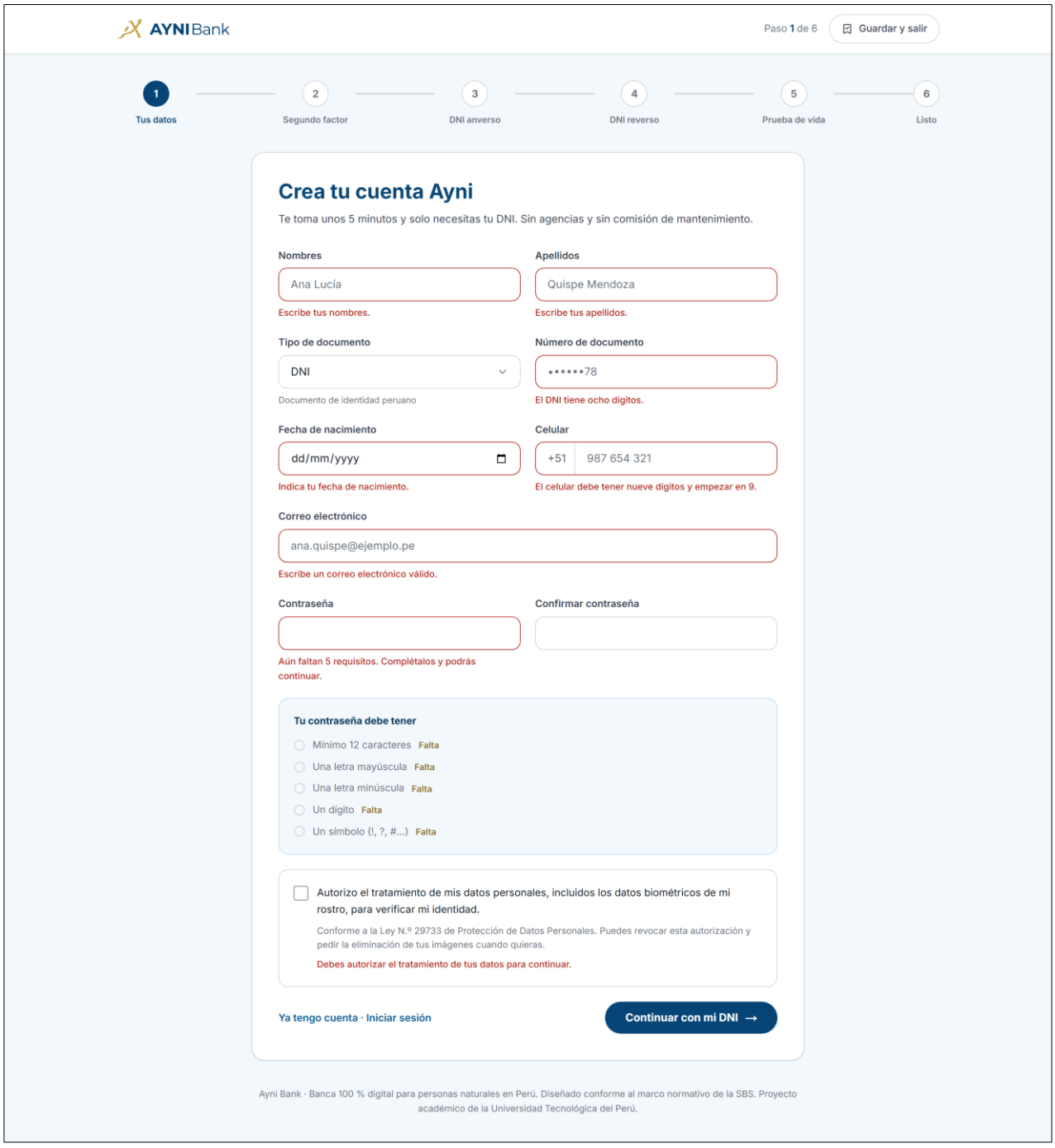
Nota. Elaboración propia a partir de `docs/calidad/reporte-usabilidad.md` del repositorio.

Figura 22 · Validación en línea: el error dice qué corregir.



Nota. Captura de pantalla de la aplicación web de Ayni Bank.

Figura 23 · Registro: los campos se marcan sin enviar nada al servidor.



Nota. Captura de pantalla de la aplicación web de Ayni Bank.

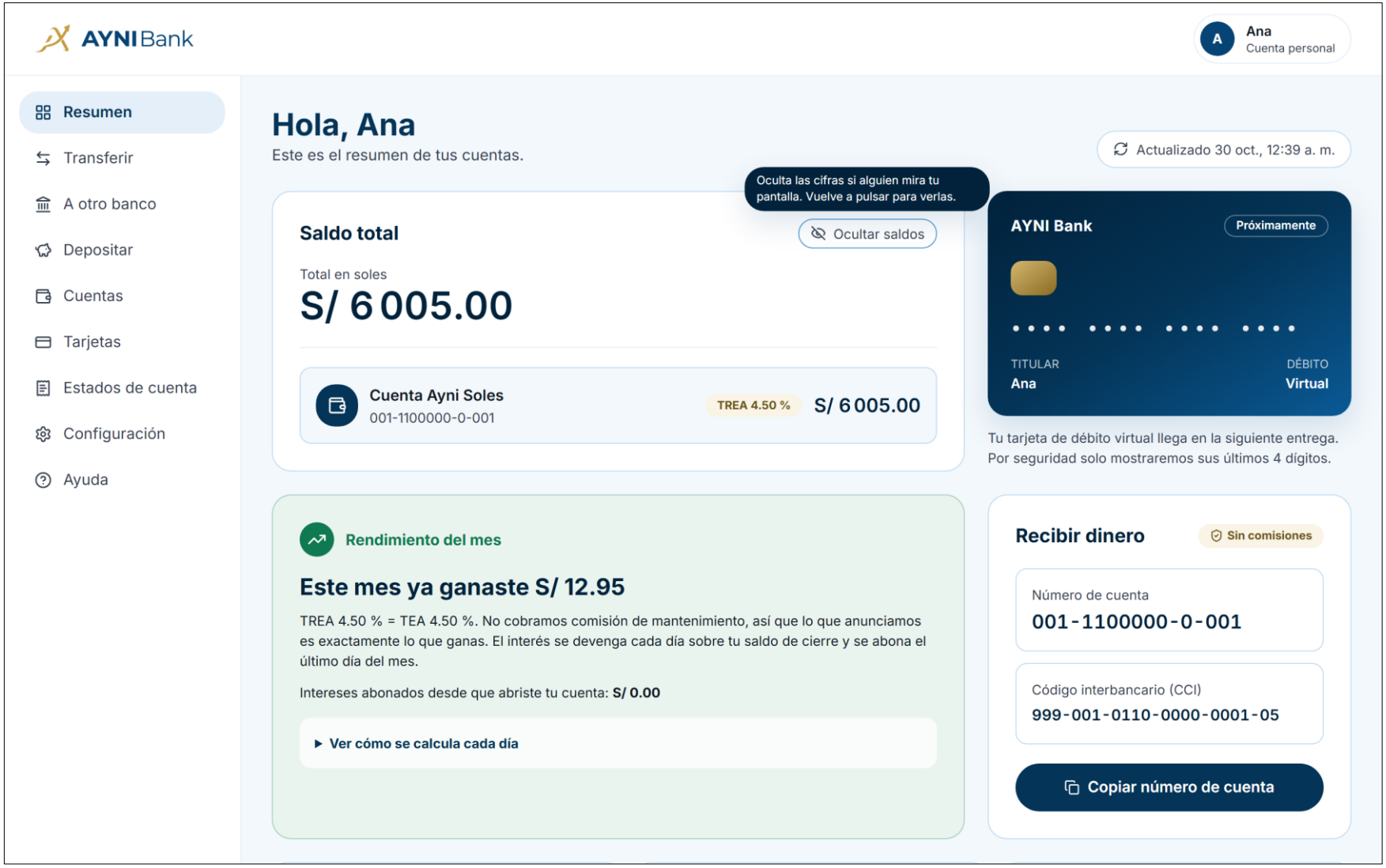
5.1.3 Asistencia al usuario

Tabla 22 · Medidas de asistencia al usuario

Medida	B	A antes	X antes	A ahora	X ahora	Nivel
Mensajes de error comprensibles	50	3	0,94	0	**1,00**	Alto
Pantallas sin violaciones de WCAG 2.1 AA	12	0	1,00	0	**1,00**	Alto

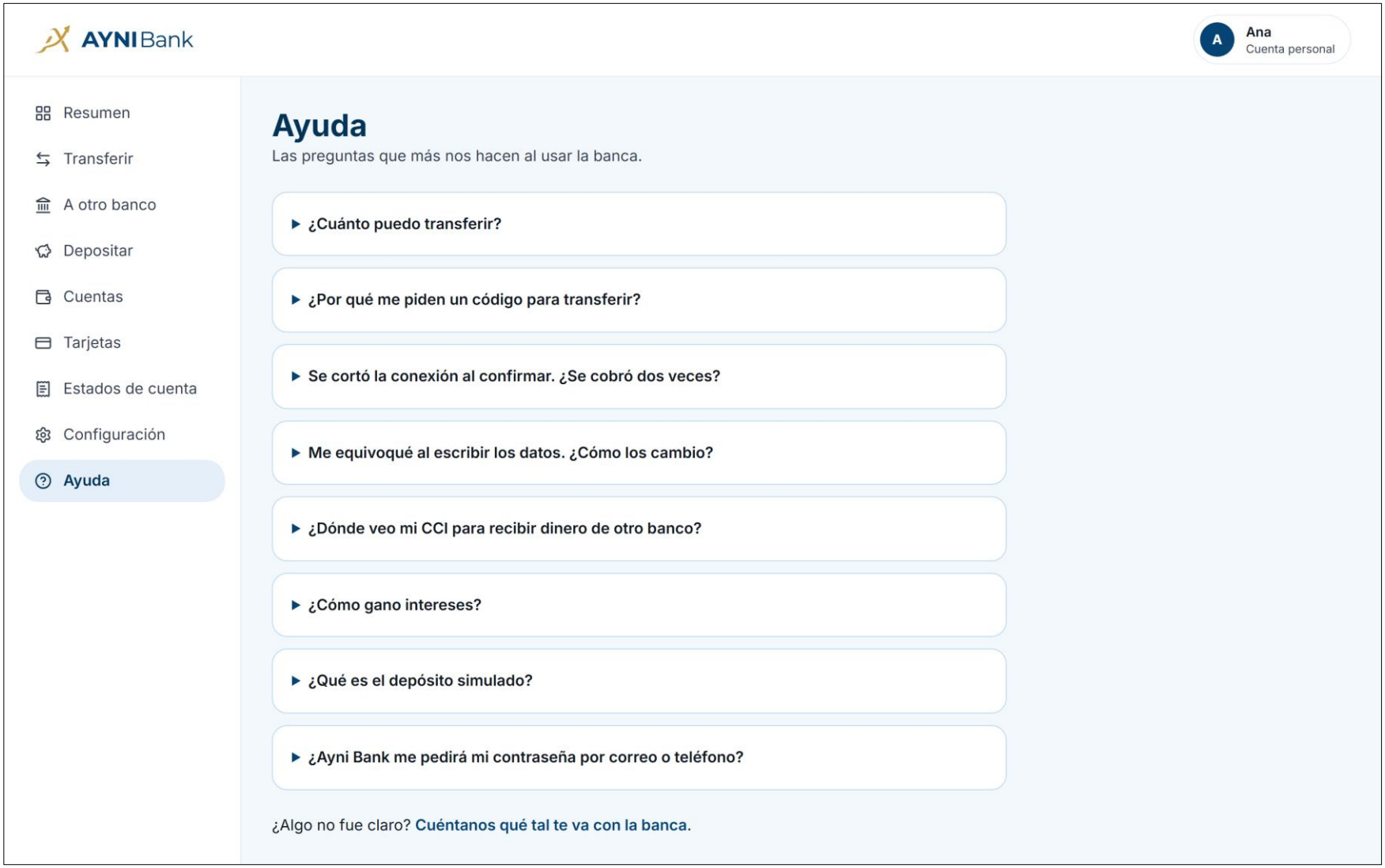
Nota. Elaboración propia a partir de `docs/calidad/reporte-usabilidad.md` del repositorio.

Figura 24 · Pista contextual en un ícono del panel.



Nota. Captura de pantalla de la aplicación web de Ayni Bank.

Figura 25 · Preguntas frecuentes, enlazadas desde cada operación.



Nota. Captura de pantalla de la aplicación web de Ayni Bank.

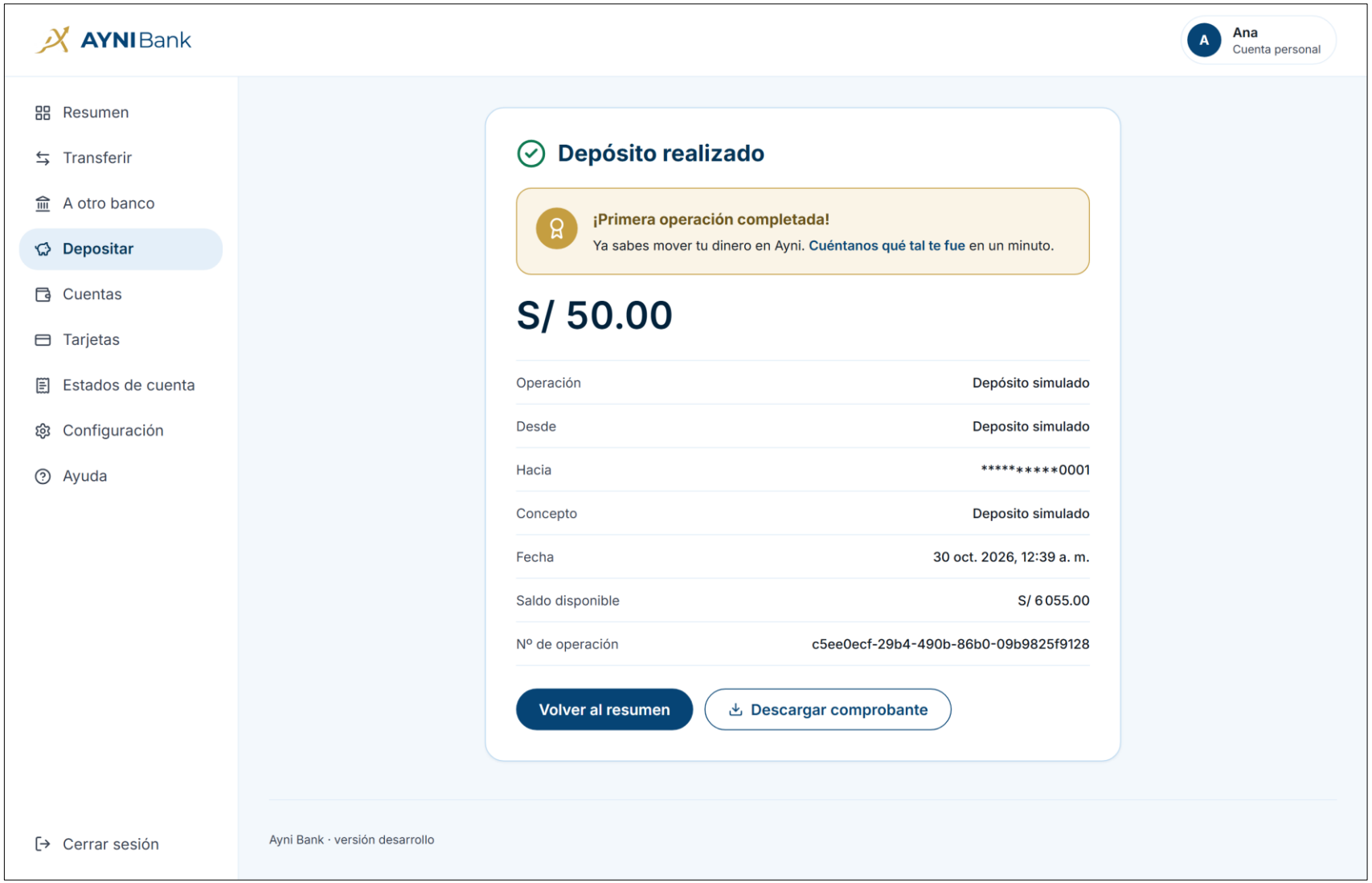
5.1.4 Compromiso del usuario

Tabla 23 · Mecanismos de compromiso y su prueba automática

Mecanismo	Cómo está en Ayni Bank	Prueba automática
Personalización	El panel saluda por el nombre: «Hola, Ana»	`panel.spec.ts`
Progreso visible en tareas de varios pasos	Registro, verificación y transferencias muestran el paso actual	`transferir.test.tsx`, `AsistenteDeBienvenida.test.tsx`
Pistas en los íconos principales, también con el teclado	Cinco en el panel, de 80 caracteres como máximo	`Pista.test.tsx`, `compromiso.spec.ts`
Asistente de bienvenida en el primer ingreso	Cuatro pasos con barra de progreso, una sola vez	`AsistenteDeBienvenida.test.tsx`, `compromiso.spec.ts`
Reconocimiento de logro	«¡Primera operación completada!» junto al primer comprobante	`LogroDePrimeraOperacion.test.tsx`
Ayuda contextual	Página «Ayuda» con preguntas frecuentes, enlazada desde cada operación	`compromiso.spec.ts`
Retroalimentación del usuario	Página «Tu opinión» con el cuestionario del anexo A	`opinion.test.tsx`

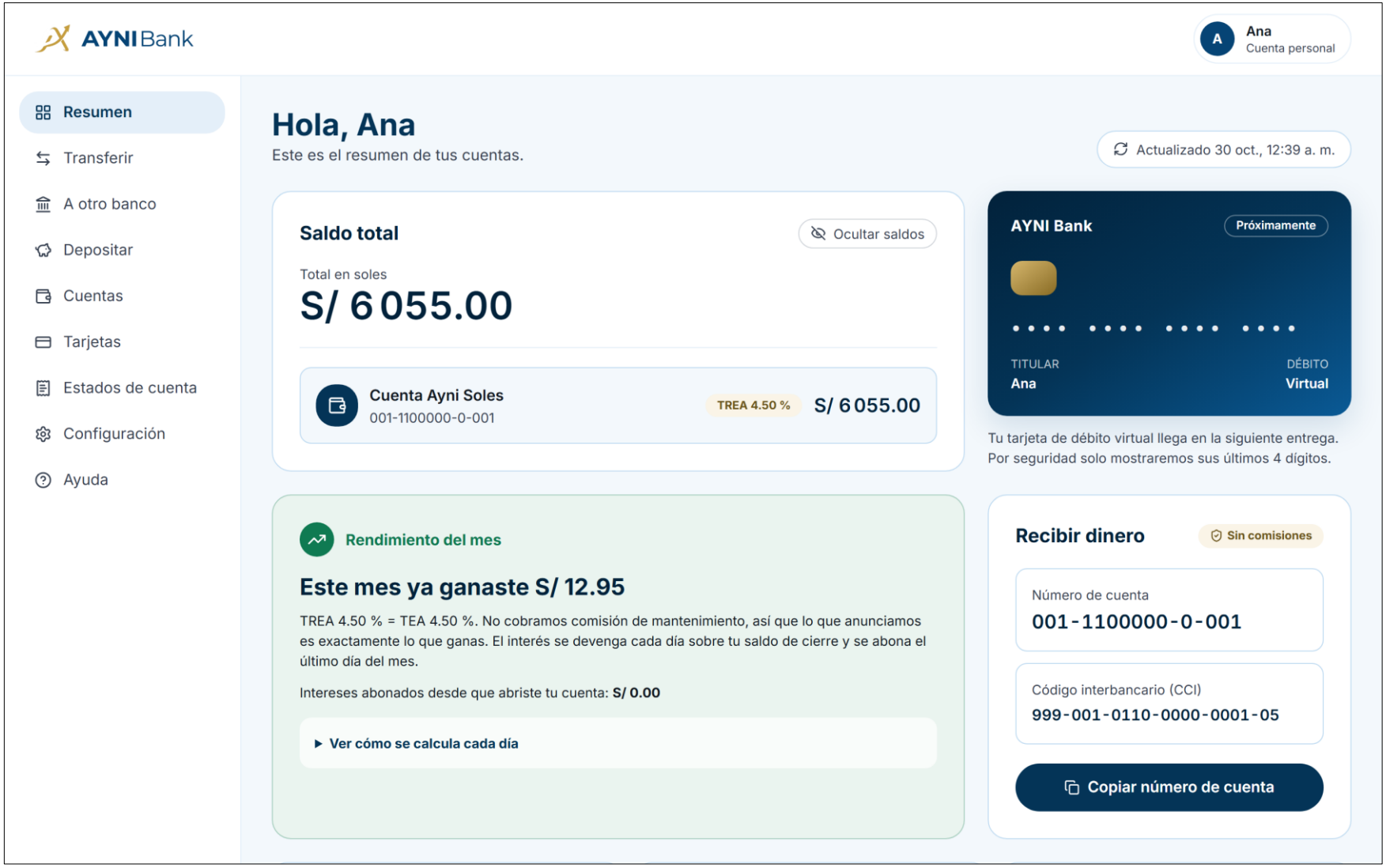
Nota. Elaboración propia a partir de `docs/calidad/reporte-usabilidad.md` del repositorio.

Figura 26 · Reconocimiento de la primera operación.



Nota. Captura de pantalla de la aplicación web de Ayni Bank.

Figura 27 · El panel muestra cuánto ganó la clienta este mes.



Nota. Captura de pantalla de la aplicación web de Ayni Bank.

5.2 Resultados

Tabla 24 · Resultado de la evaluación de usabilidad

Subcaracterística	Medida principal	X antes	X ahora	Nivel
Facilidad de aprendizaje	Formatos anunciados de antemano	0,91	1,00	**Alto**
Protección contra errores	Acciones sensibles protegidas	0,90	1,00	**Alto**
Asistencia al usuario	Mensajes de error comprensibles	0,94	1,00	**Alto**
	Pantallas sin violaciones de WCAG 2.1 AA	1,00	1,00	**Alto**
Compromiso del usuario	Mecanismos de compromiso implementados y verificados	—	1,00	**Alto**

Nota. Elaboración propia a partir de `docs/calidad/reporte-usabilidad.md` del repositorio.

5.3 Conclusiones

- La versión 2.0.0 alcanza el nivel **Alto** en las cuatro subcaracterísticas: aprende sola, previene los errores evitables, explica cómo corregir los demás y ofrece los mecanismos que motivan a seguir usándola.
- La evaluación encontró y corrigió **diez defectos de usabilidad**, entre ellos un depósito que podía duplicarse al reintentar y tres mensajes que no decían qué hacer.
- La percepción de los usuarios (cuestionario SUS con cinco participantes) se medirá en el proyecto final.

6. DESPLIEGUE DE LA VERSIÓN 2 (CAP. 11, ACTUALIZADO)

6.1 Manual de despliegue, versión 2

El manual (`docs/despliegue/manual-de-despliegue.md`) se actualizó con la versión 2: los secretos nuevos, Redis, Amazon SES, la cámara interbancaria, el procedimiento de la release y la validación contra la nube. La tabla resume qué cambió y dónde se configura cada pieza que integra un sistema externo.

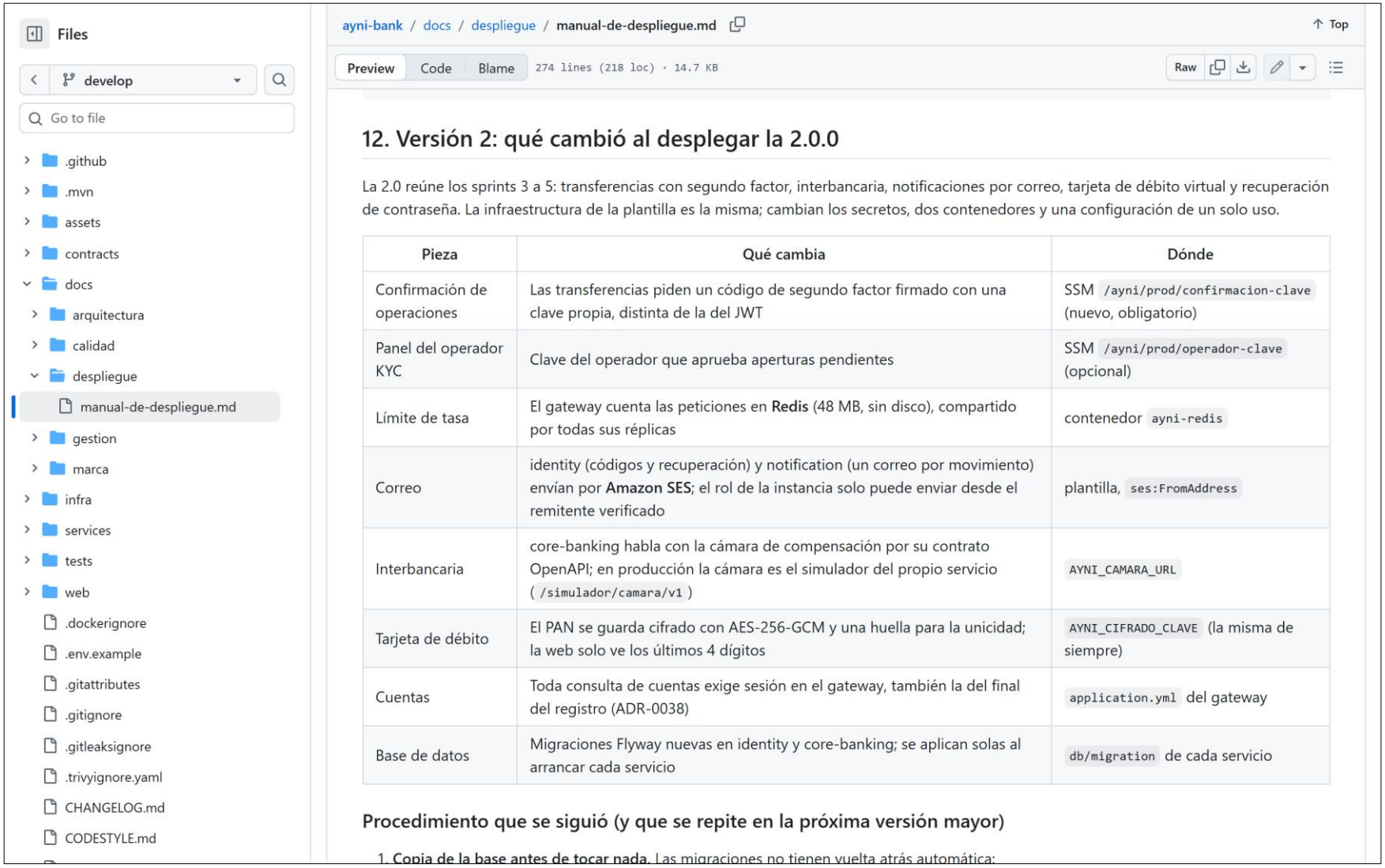
Tabla 25 · Qué cambió al desplegar la versión 2.0.0

Pieza	Qué cambia	Dónde
Confirmación de operaciones	Las transferencias piden un código de segundo factor firmado con una clave propia, distinta de la del JWT	SSM `/ayni/prod/confirmacion-clave` (nuevo, obligatorio)
Panel del operador KYC	Clave del operador que aprueba aperturas pendientes	SSM `/ayni/prod/operador-clave` (opcional)
Límite de tasa	El gateway cuenta las peticiones en **Redis** (48 MB, sin disco), compartido por todas sus réplicas	contenedor `ayni-redis`
Correo	identity (códigos y recuperación) y notification (un correo por movimiento) envían por **Amazon SES**; el rol de la instancia solo puede enviar desde el remitente verificado	plantilla, `ses:FromAddress`
Interbancaria	core-banking habla con la cámara de compensación por su contrato OpenAPI; en producción la cámara es el simulador del propio servicio (`/simulador/camara/v1`)	`AYNI_CAMARA_URL`
Tarjeta de débito	El PAN se guarda cifrado con AES-256-GCM y una huella para la unicidad; la web solo ve los últimos 4 dígitos	`AYNI_CIFRADO_CLAVE` (la misma de siempre)
Cuentas	Toda consulta de cuentas exige sesión en el gateway, también la del final del registro (ADR-0038)	`application.yml` del gateway
Base de datos	Migraciones Flyway nuevas en identity y core-banking; se aplican solas al arrancar cada servicio	`db/migration` de cada servicio

Nota. Elaboración propia a partir de `docs/despliegue/manual-de-despliegue.md` del repositorio.

- **Copia de la base** antes de tocar nada: snapshot de RDS `ayni-bank-prod-antes-de-2-0-0`.
- **Secretos nuevos** en Parameter Store (`confirmacion-clave`); sin ellos el despliegue se detiene.
- **Release** `release/2.0.0` → `main` con la CI en verde y la aprobación de la revisora: el CD publica las imágenes en ECR, despliega por SSM, pasa las pruebas de humo y crea el tag `v2.0.0`.
- **Republicación de titulares** una sola vez, para que el servicio de notificaciones conozca a los clientes existentes.
- Si algo falla: reversión automática de imágenes, restauración del snapshot o *hotfix* a `main`.

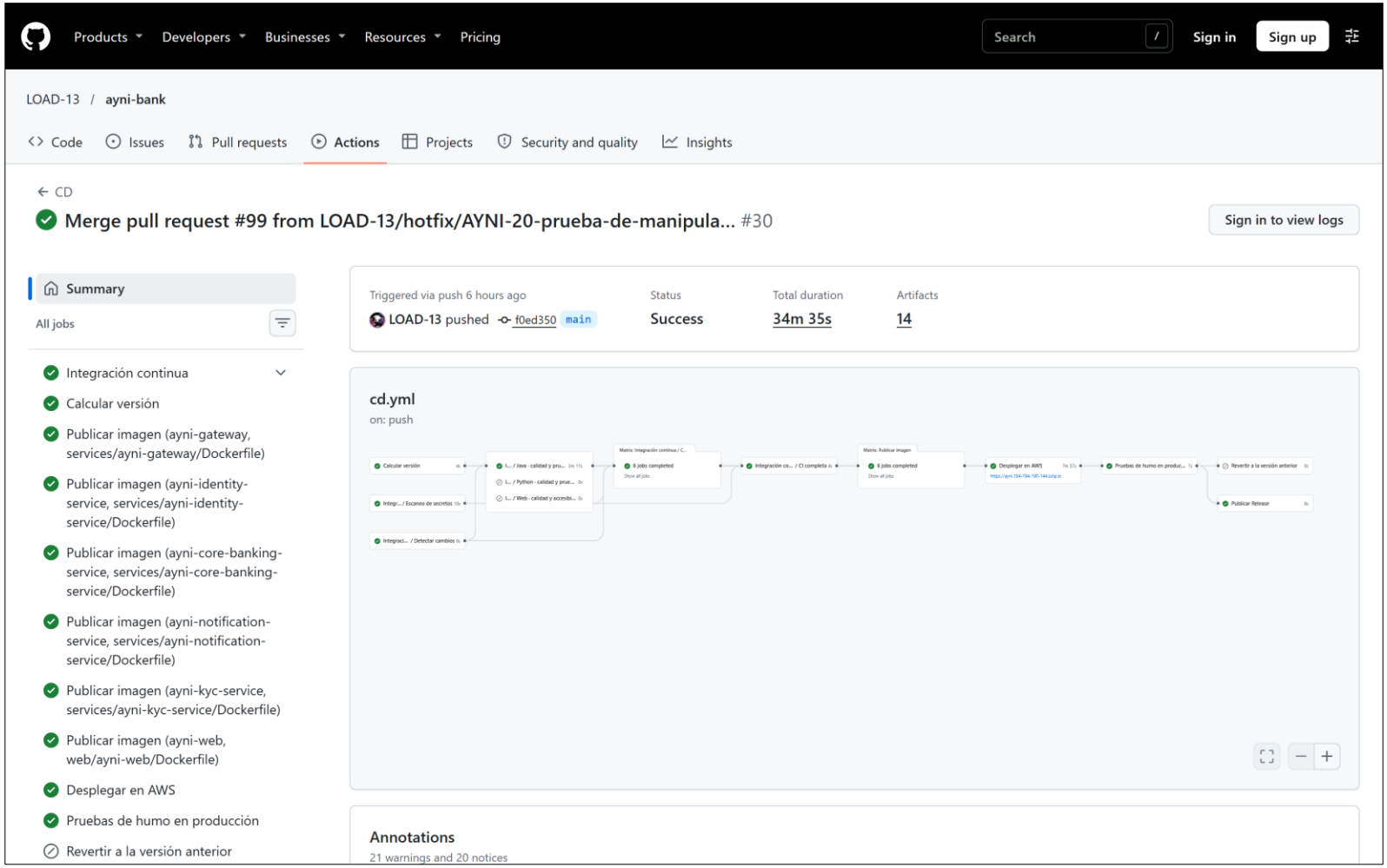
Figura 28 · La sección 12 del manual de despliegue en el repositorio.



Nota. Captura de docs/despliegue/manual-de-despliegue.md en GitHub.

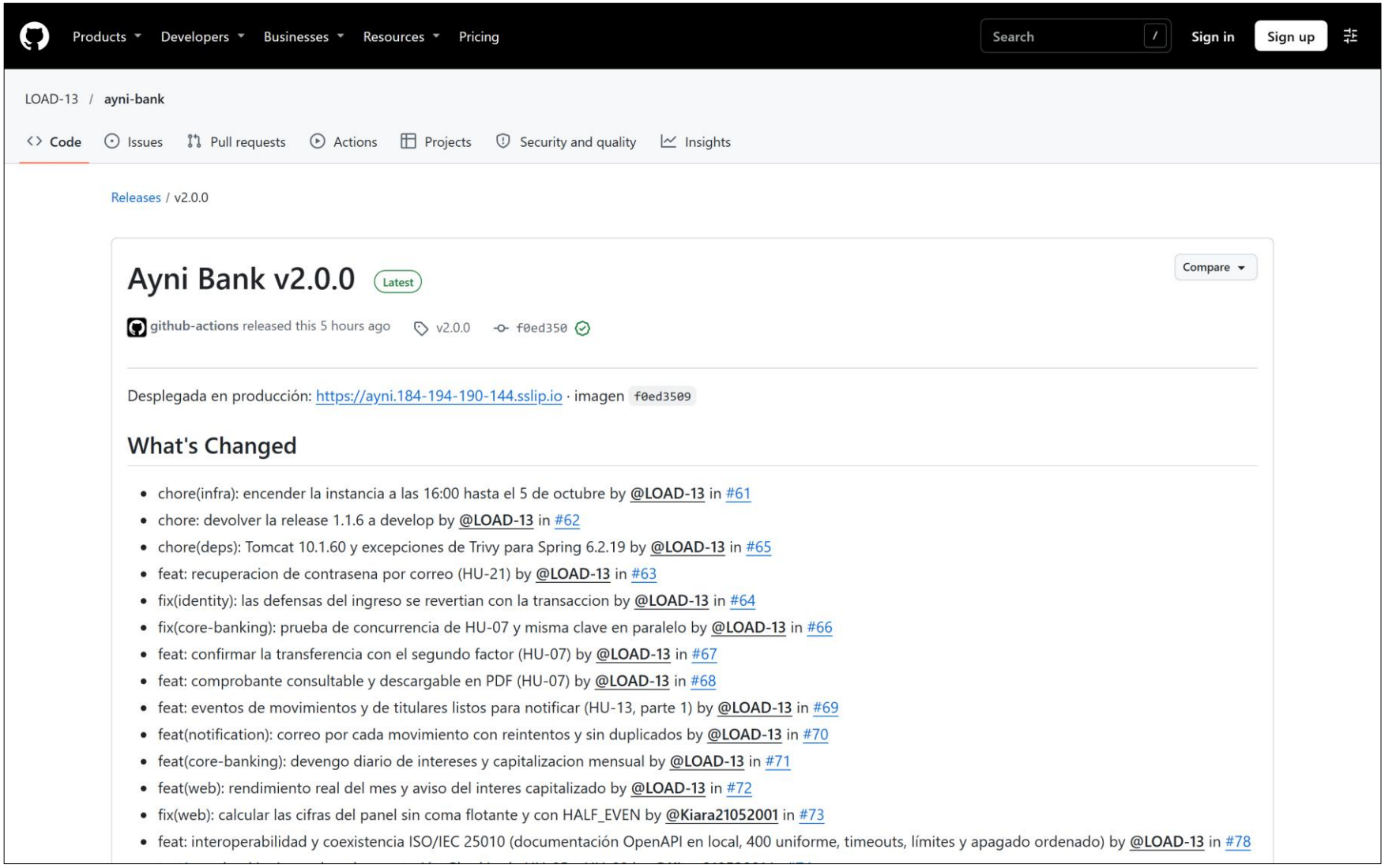
6.2 Evidencia de pruebas de despliegue

Figura 29 · El CD de la versión 2.0.0 en GitHub Actions: despliegue y pruebas de humo en verde.



Nota. Captura de GitHub Actions: flujo cd.yml sobre la rama main.

Figura 30 · Release v2.0.0 publicada por el pipeline, con su registro de cambios.



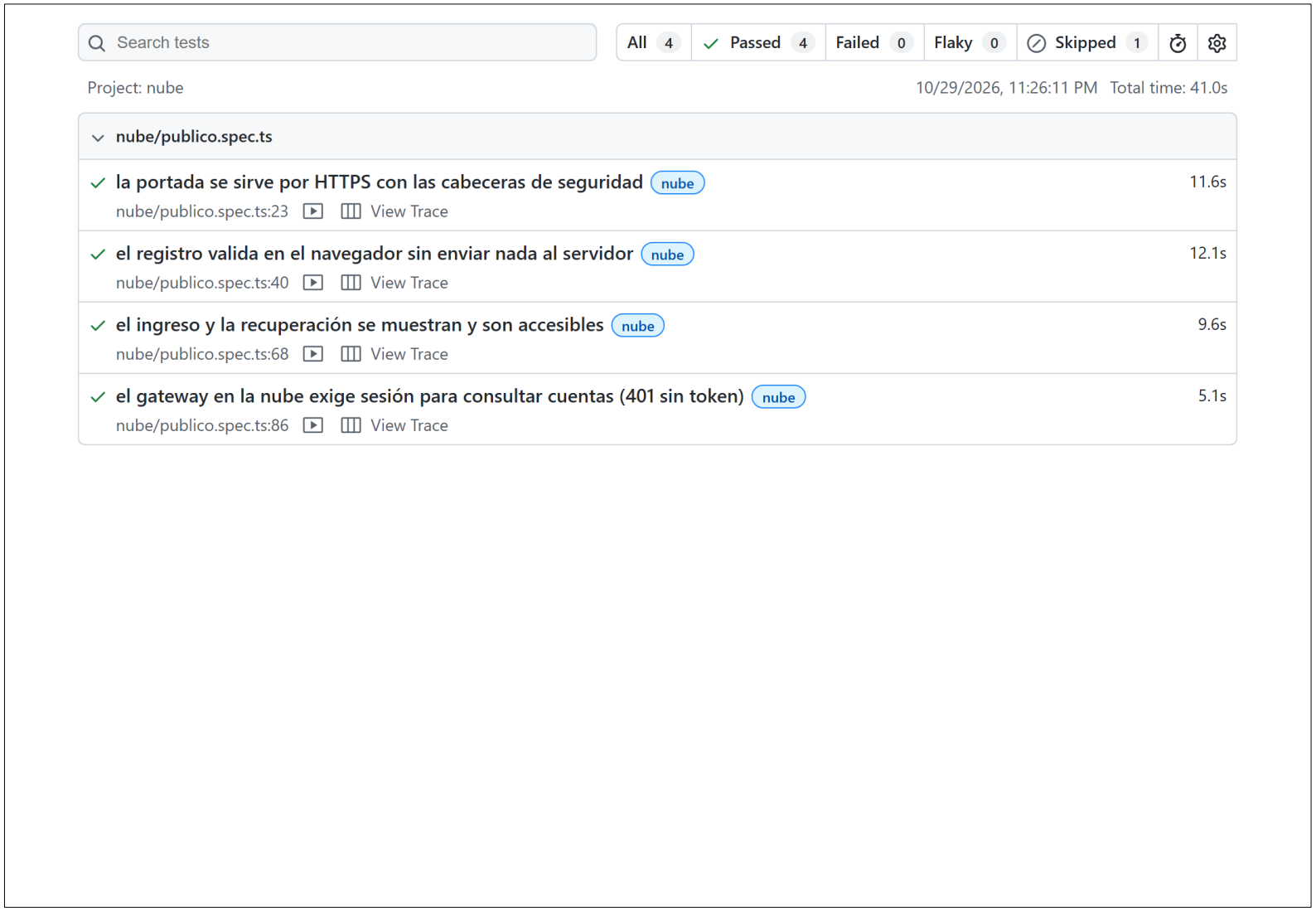
Nota. Captura de la página de releases del repositorio en GitHub.

Tabla 26 · Comprobaciones sobre la versión desplegada

Comprobación (2.0.0, 30-oct-2026)	Resultado
CD de `main` (imágenes, despliegue, humo, tag)	verde, tag `v2.0.0`
Web por HTTPS y versión en el ingreso	200, «versión v2.0.0»
HSTS, CSP con `frame-ancestors 'none'`, `X-Frame-Options: DENY`, `nosniff`	presentes
Registro, ingreso y recuperación sin violaciones WCAG 2.1 AA (axe)	0 violaciones
`GET /api/v1/cuentas/mia` sin token	401

Nota. Elaboración propia a partir de `docs/despliegue/manual-de-despliegue.md` del repositorio.

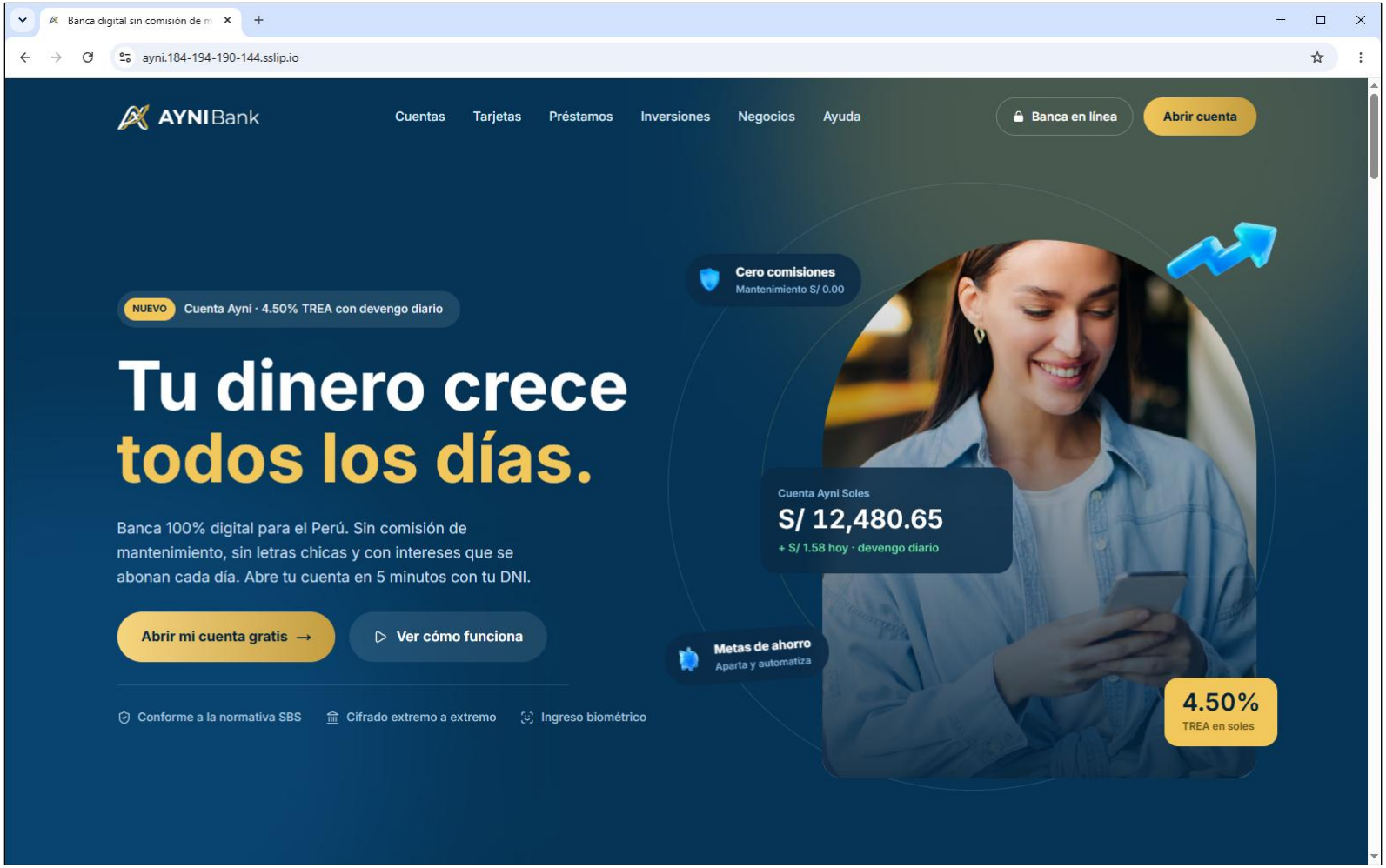
Figura 31 · La suite de Playwright contra AWS: cuatro de cuatro en verde.



Nota. Captura del reporte HTML de Playwright, proyecto «nube».

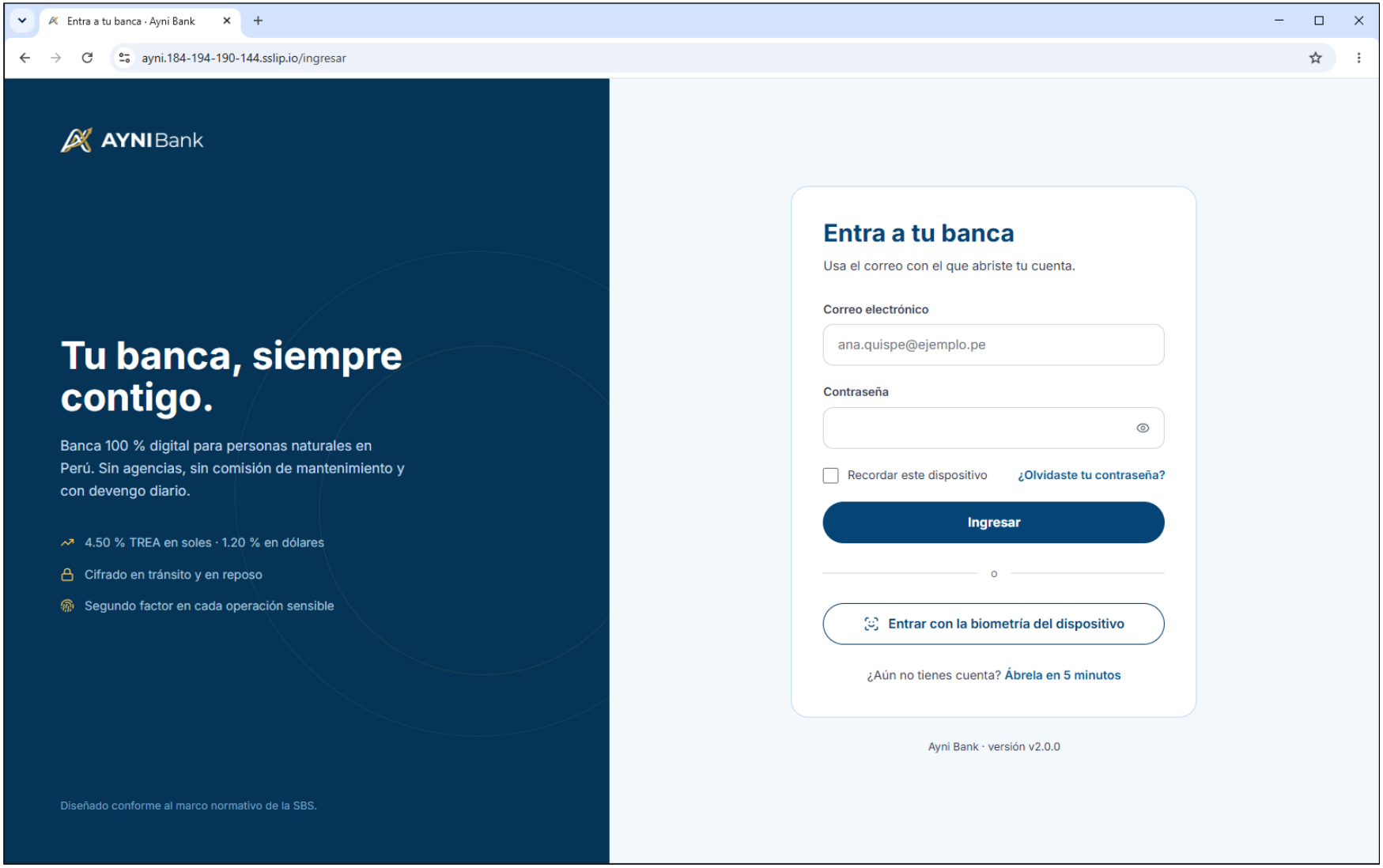
6.3 El sistema funcionando en AWS

Figura 32 · La portada servida por HTTPS desde AWS.



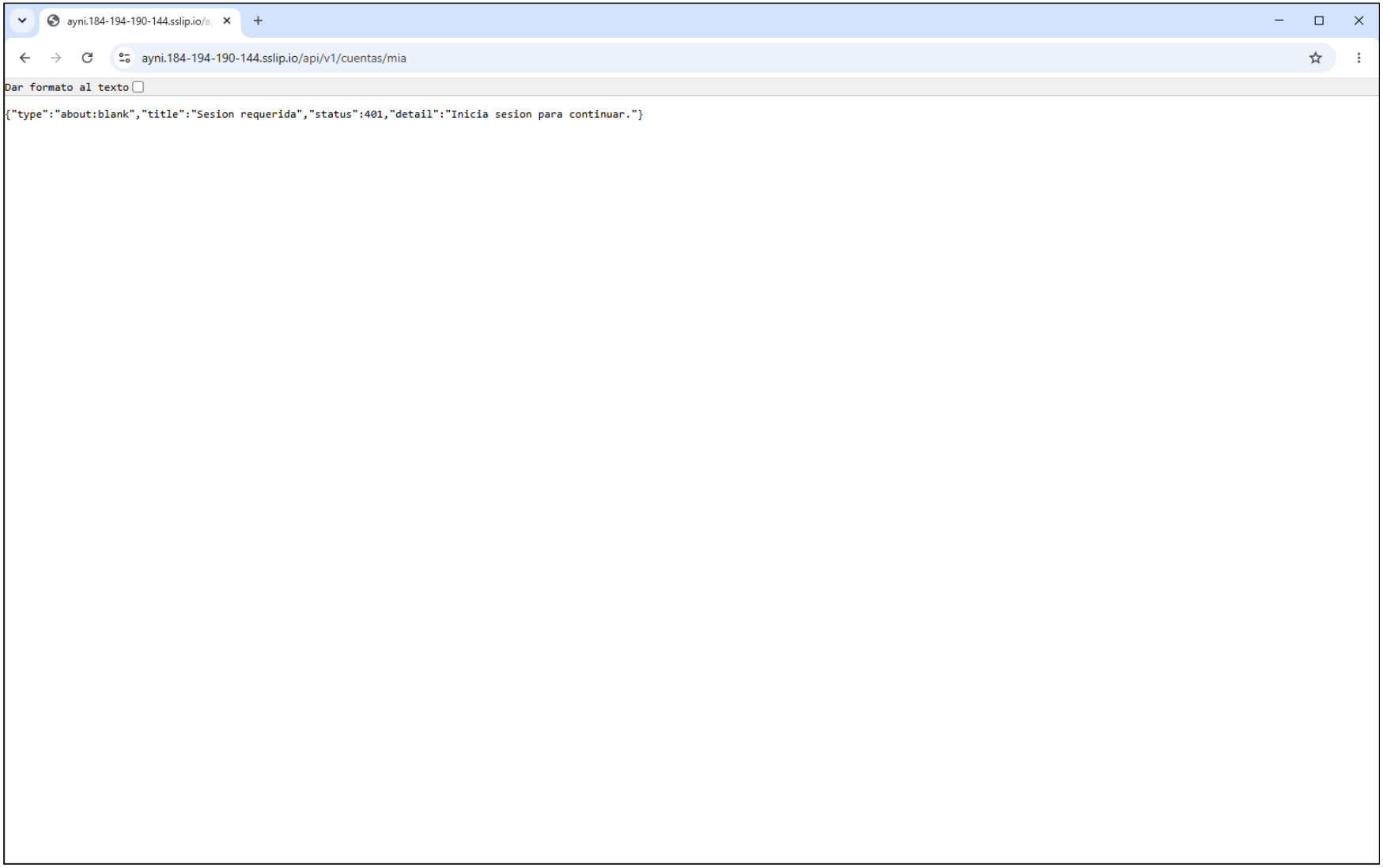
Nota. Captura del navegador en https://ayni.produccion.ejemplo.

Figura 33 · La pantalla de ingreso muestra la versión publicada: v2.0.0.



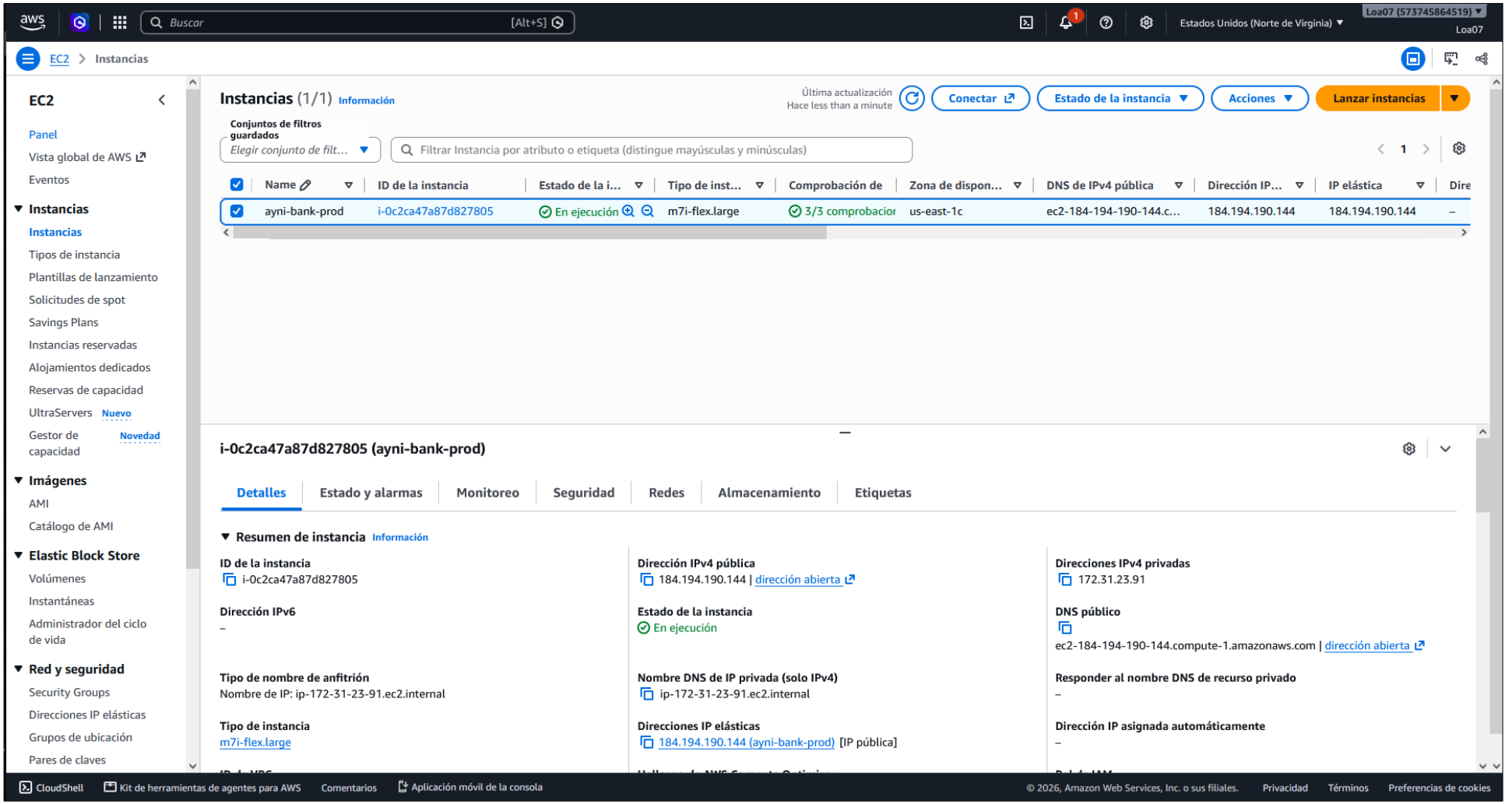
Nota. Captura del navegador en https://ayni.produccion.ejemplo/ingresar.

Figura 34 · Sin sesión, el gateway en la nube responde 401.



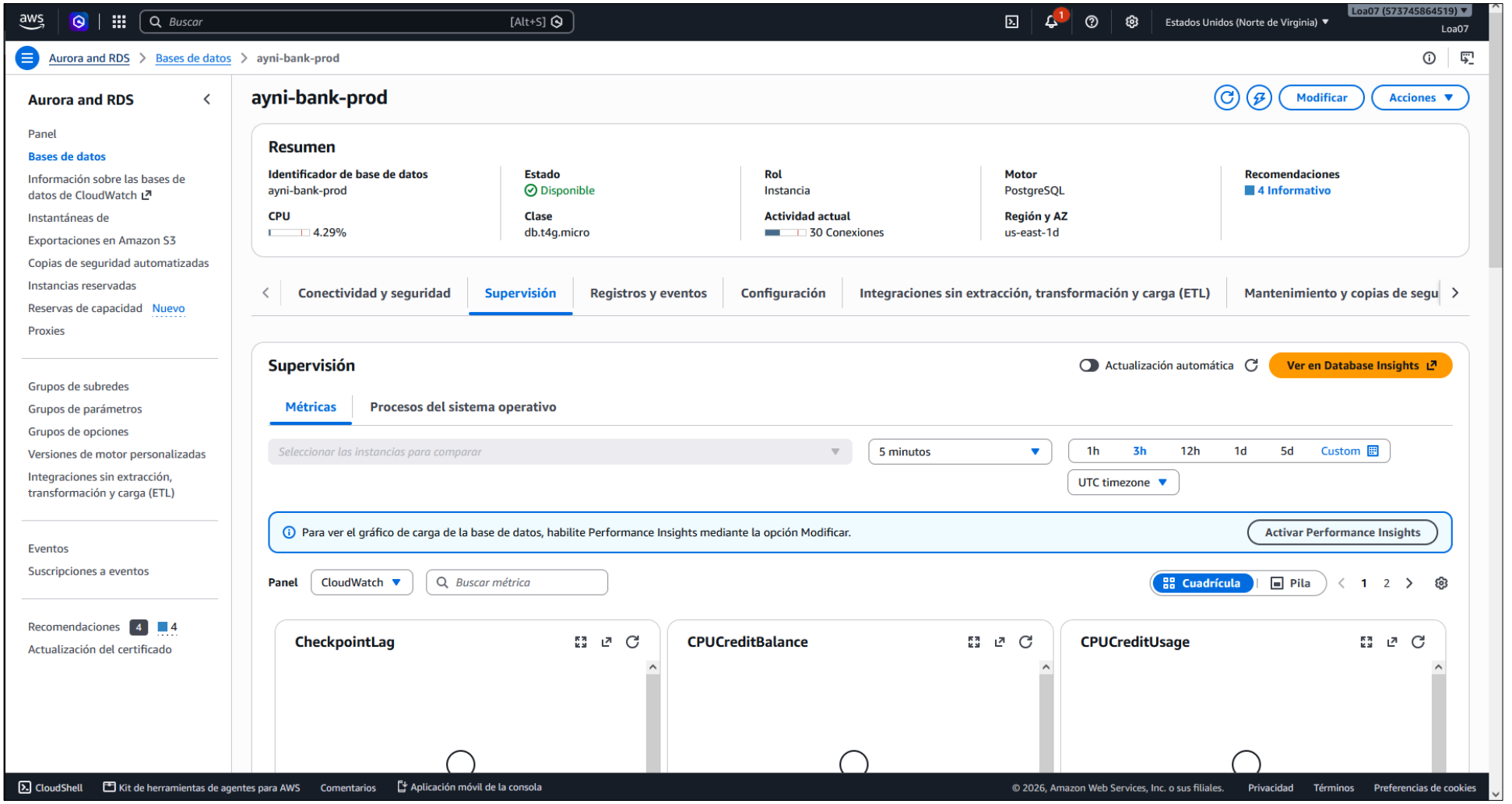
Nota. Captura del navegador en https://ayni.produccion.ejemplo/api/v1/cuentas/mia.

Figura 35 · Amazon EC2: la instancia de producción en ejecución.



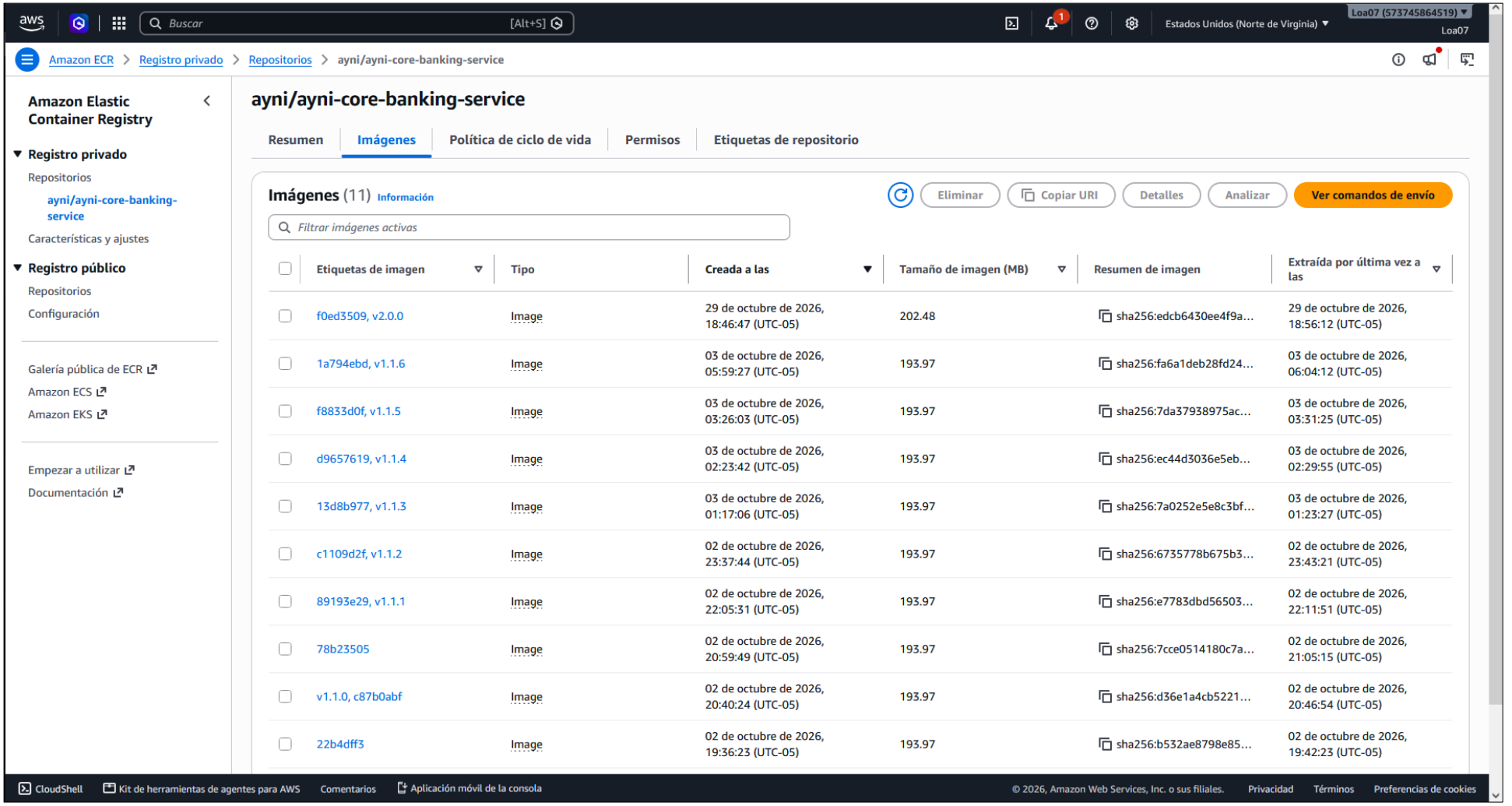
Nota. Captura de la consola de AWS: Amazon EC2.

Figura 36 · Amazon RDS: la base de datos PostgreSQL disponible.



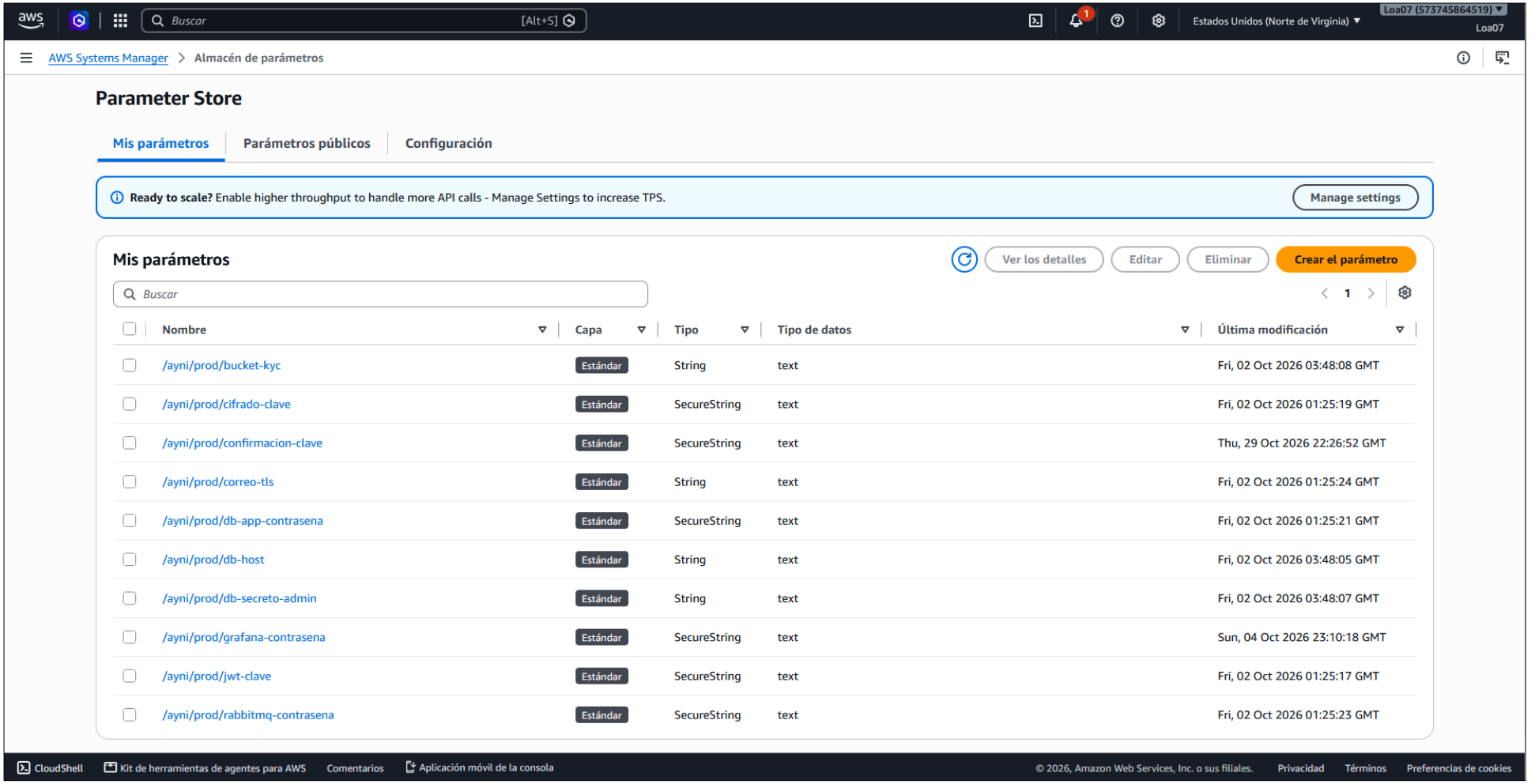
Nota. Captura de la consola de AWS: Amazon RDS.

Figura 37 · Amazon ECR: las imágenes de cada servicio, la última v2.0.0.



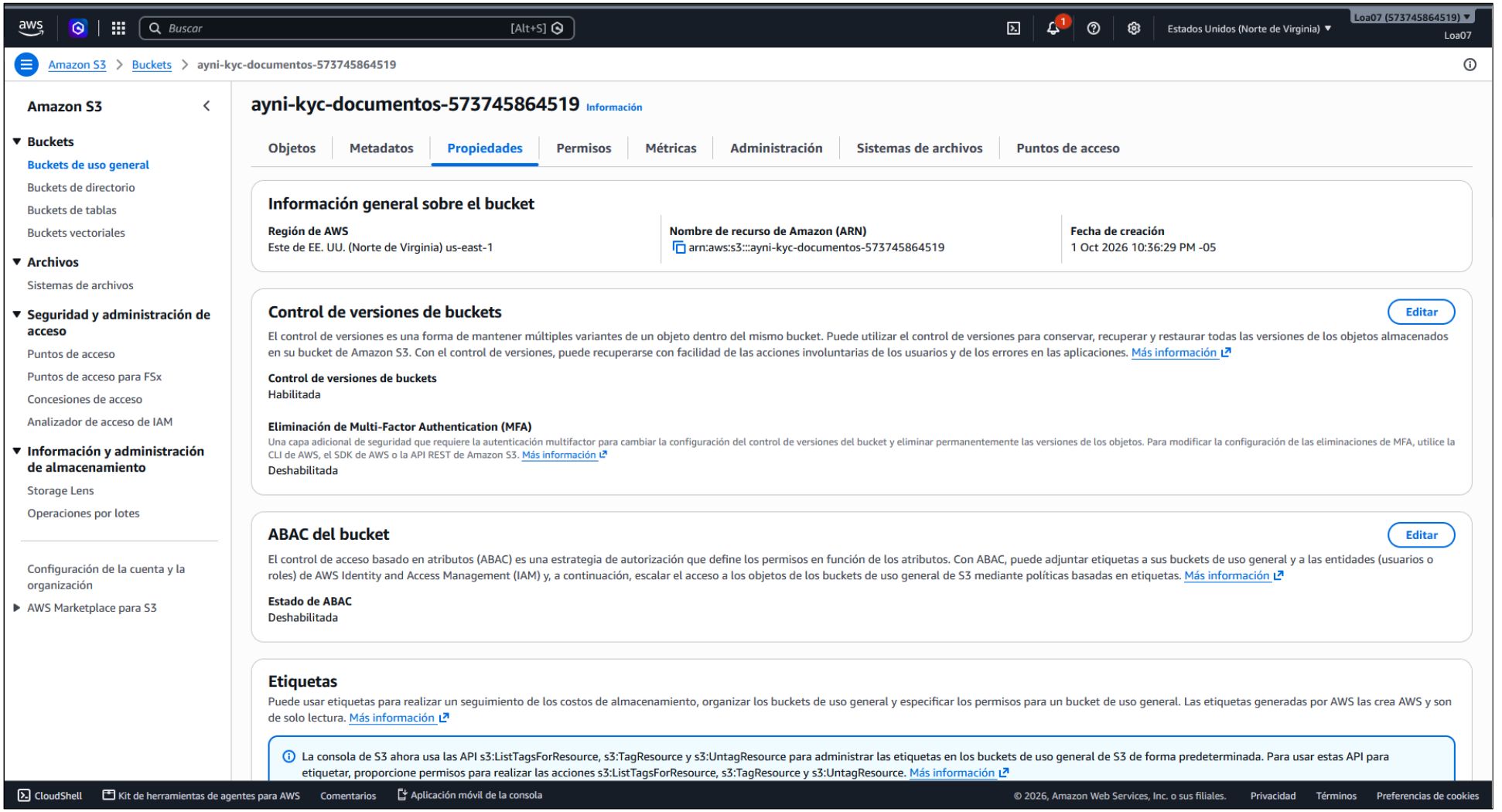
Nota. Captura de la consola de AWS: Amazon ECR.

Figura 38 · Parameter Store: los secretos, cifrados, fuera del repositorio.



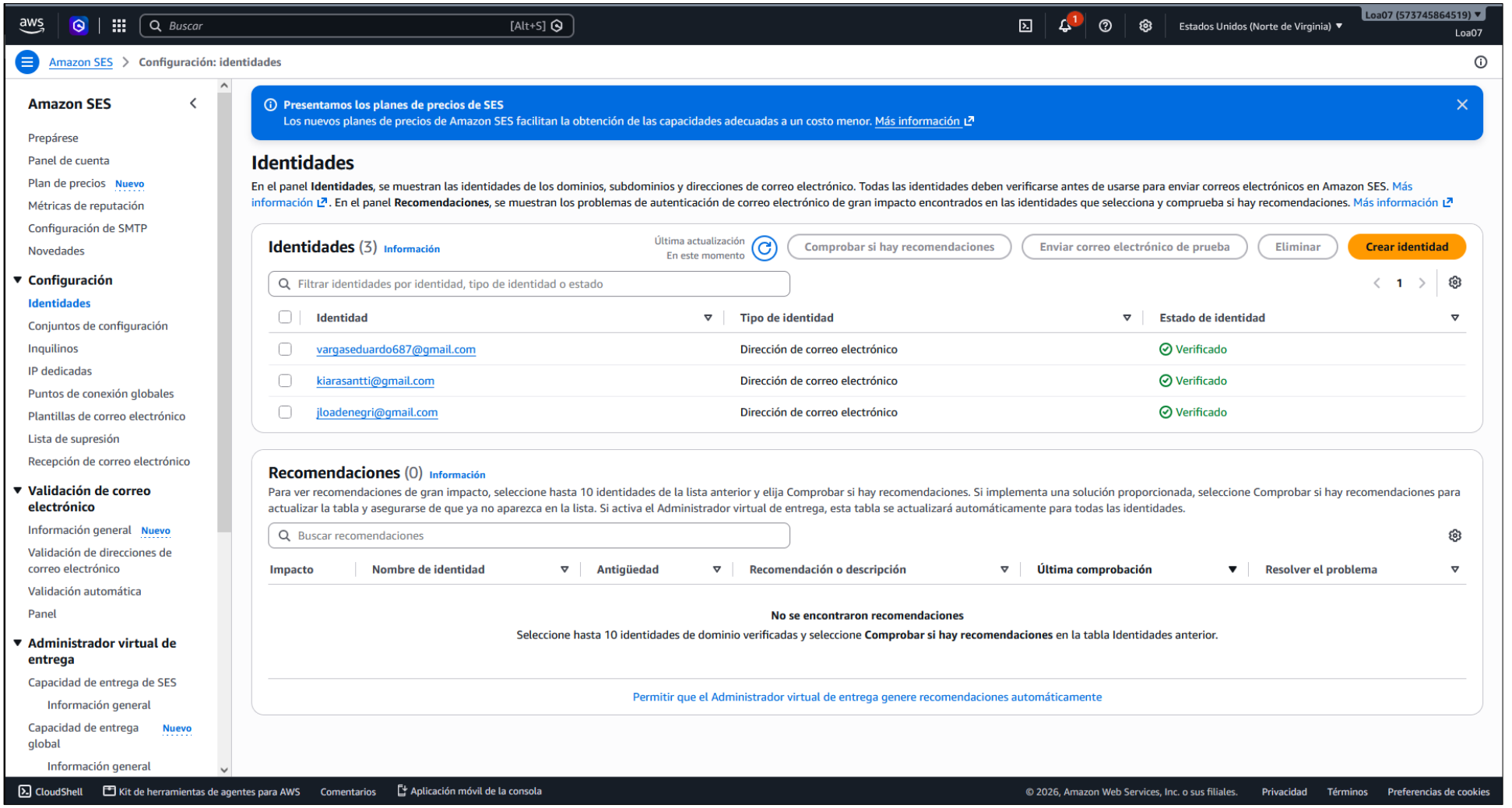
Nota. Captura de la consola de AWS: Systems Manager, Parameter Store.

Figura 39 · Amazon S3: el bucket de documentos KYC, privado y con versiones.



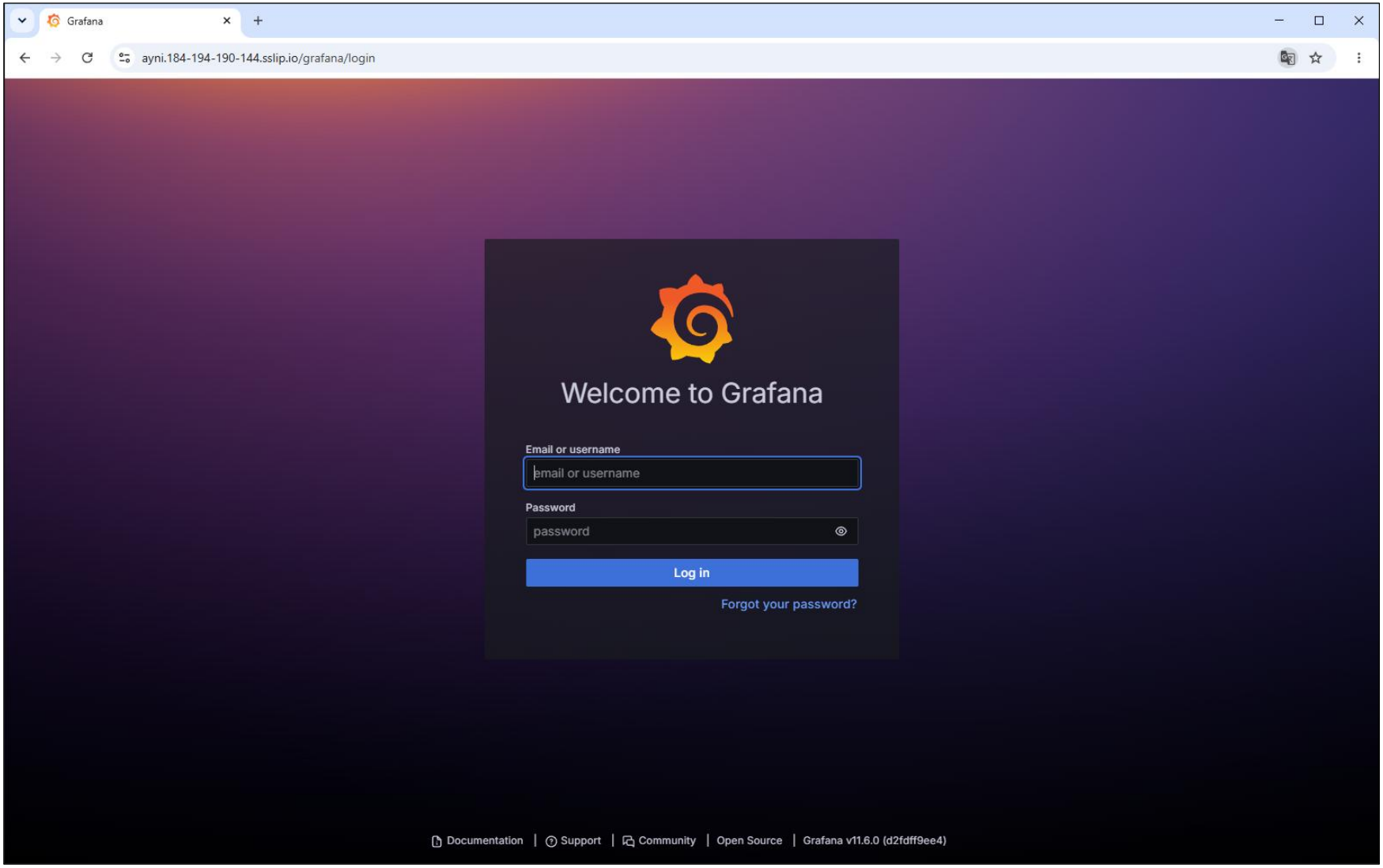
Nota. Captura de la consola de AWS: Amazon S3.

Figura 40 · Amazon SES: identidades verificadas para el envío de correos.



Nota. Captura de la consola de AWS: Amazon SES.

Figura 41 · Grafana publicado en la nube, con acceso por contraseña.



Nota. Captura del navegador en https://ayni.produccion.ejemplo/grafana/login.

7. HISTORIAL DE COMMITS (ANEXO H)

El historial se obtiene de git con un script (`infra/scripts/evidencia-de-commits.py`); no se edita a mano. Todo cambio entra por pull request, con la CI en verde y la aprobación de la revisora, siguiendo GitFlow.

Tabla 27 · Resumen del historial de commits

Periodo	15-08-2026 → 30-10-2026
Commits (sin contar los de merge)	300
Pull requests fusionados	95
Commits de merge	136
Mensajes con Conventional Commits (`tipo(ámbito): …`)	300 de 300 (100,0 %)
Commits con referencia a Jira (`Refs: AYNI-NN`)	177 de 300 (59,0 %)

Nota. Elaboración propia a partir de `docs/gestion/anexo-h-historial-de-commits.md` del repositorio.

Tabla 28 · Commits por autor

Autor	Commits	%	Primer commit	Último commit	PR en los que participa
Joaquín Loa Denegri	175	58,3 %	15-08-2026 00:04	30-10-2026 03:10	77
Eduardo Vargas Zumaeta	64	21,3 %	06-09-2026 17:57	28-10-2026 23:32	8
Kiara Santti Saavedra	27	9,0 %	20-10-2026 01:39	27-10-2026 02:41	10
Gerardo Raúl Socualaya Mandamiento	15	5,0 %	26-10-2026 23:03	27-10-2026 00:18	6
Fabian	11	3,7 %	27-10-2026 18:01	28-10-2026 03:16	2
Frank Grheg Sotomayor Suasnabar	8	2,7 %	28-10-2026 19:05	28-10-2026 19:20	4

Nota. Elaboración propia a partir de `docs/gestion/anexo-h-historial-de-commits.md` del repositorio.

Tabla 29 · Distribución del trabajo por tipo de rama (GitFlow)

Tipo de rama	Ramas fusionadas	Commits que aportaron
`feature`	46	211
`bugfix`	10	19
`release`	9	176
`chore`	8	16
`fix`	7	17
`otra`	5	135
`hotfix`	4	5
`docs`	3	3
`test`	2	4
`ci`	1	2

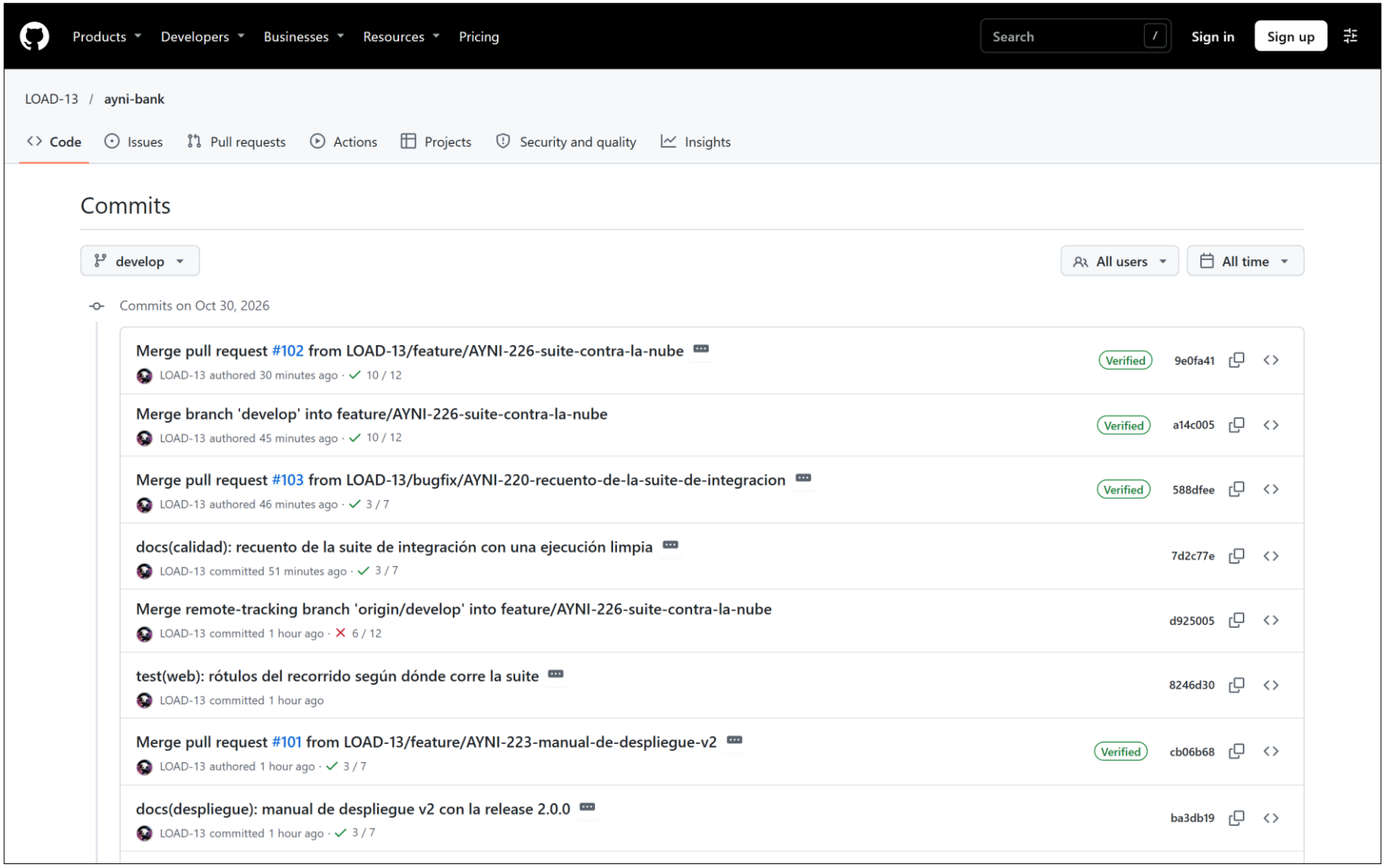
Nota. Elaboración propia a partir de `docs/gestion/anexo-h-historial-de-commits.md` del repositorio.

Tabla 30 · Mensajes por tipo (Conventional Commits)

Tipo	Commits	%
`feat`	107	35,7 %
`fix`	90	30,0 %
`docs`	35	11,7 %
`test`	33	11,0 %
`chore`	16	5,3 %
`ci`	8	2,7 %
`style`	4	1,3 %
`build`	4	1,3 %
`revert`	1	0,3 %
`refactor`	1	0,3 %
`perf`	1	0,3 %

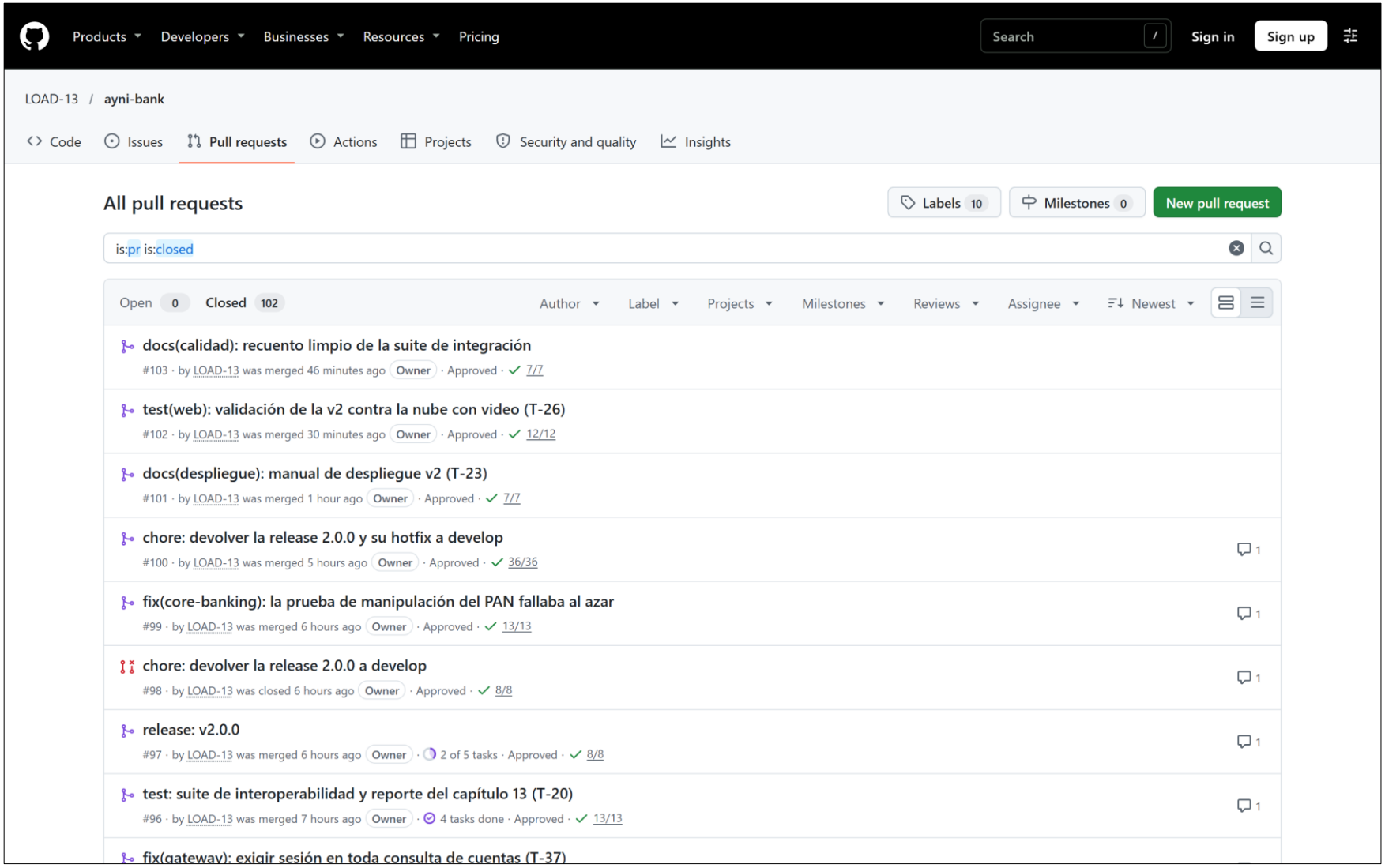
Nota. Elaboración propia a partir de `docs/gestion/anexo-h-historial-de-commits.md` del repositorio.

Figura 42 · Commits de la rama develop en GitHub.



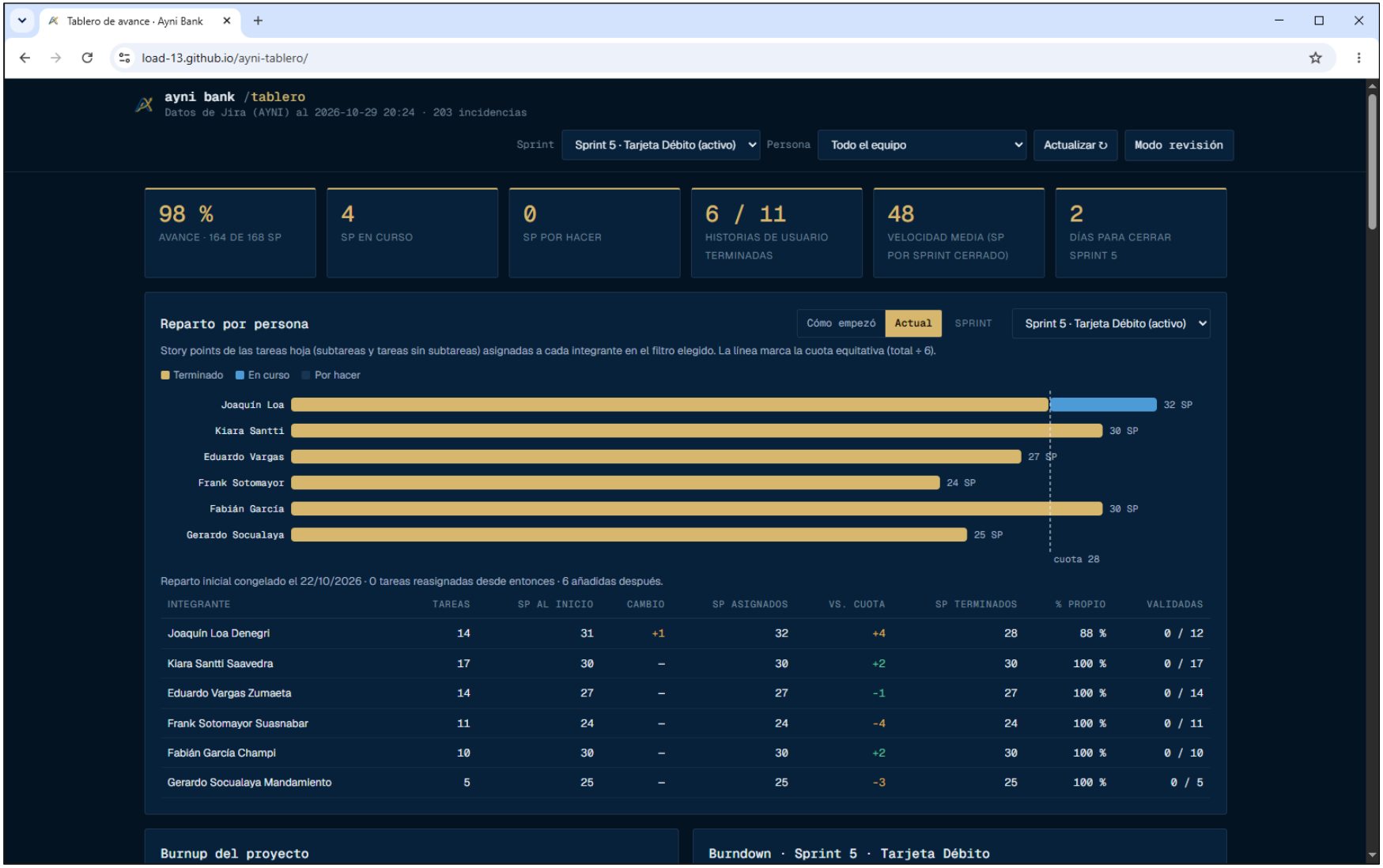
Nota. Captura de la lista de commits de la rama develop en GitHub.

Figura 43 · Pull requests fusionados, cada uno con su revisión.



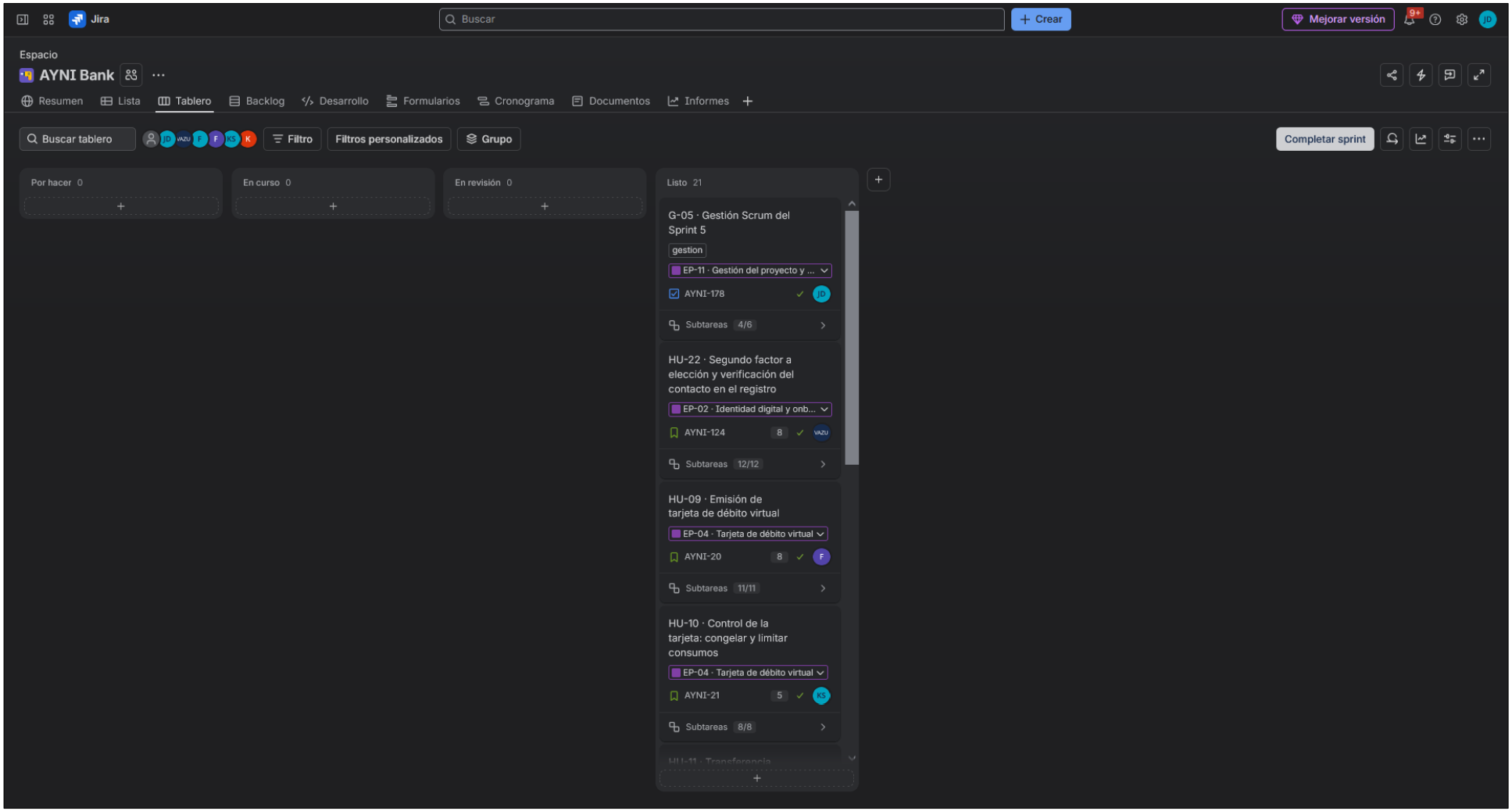
Nota. Captura de la lista de pull requests cerrados en GitHub.

Figura 44 · Tablero de reparto del Sprint 5.



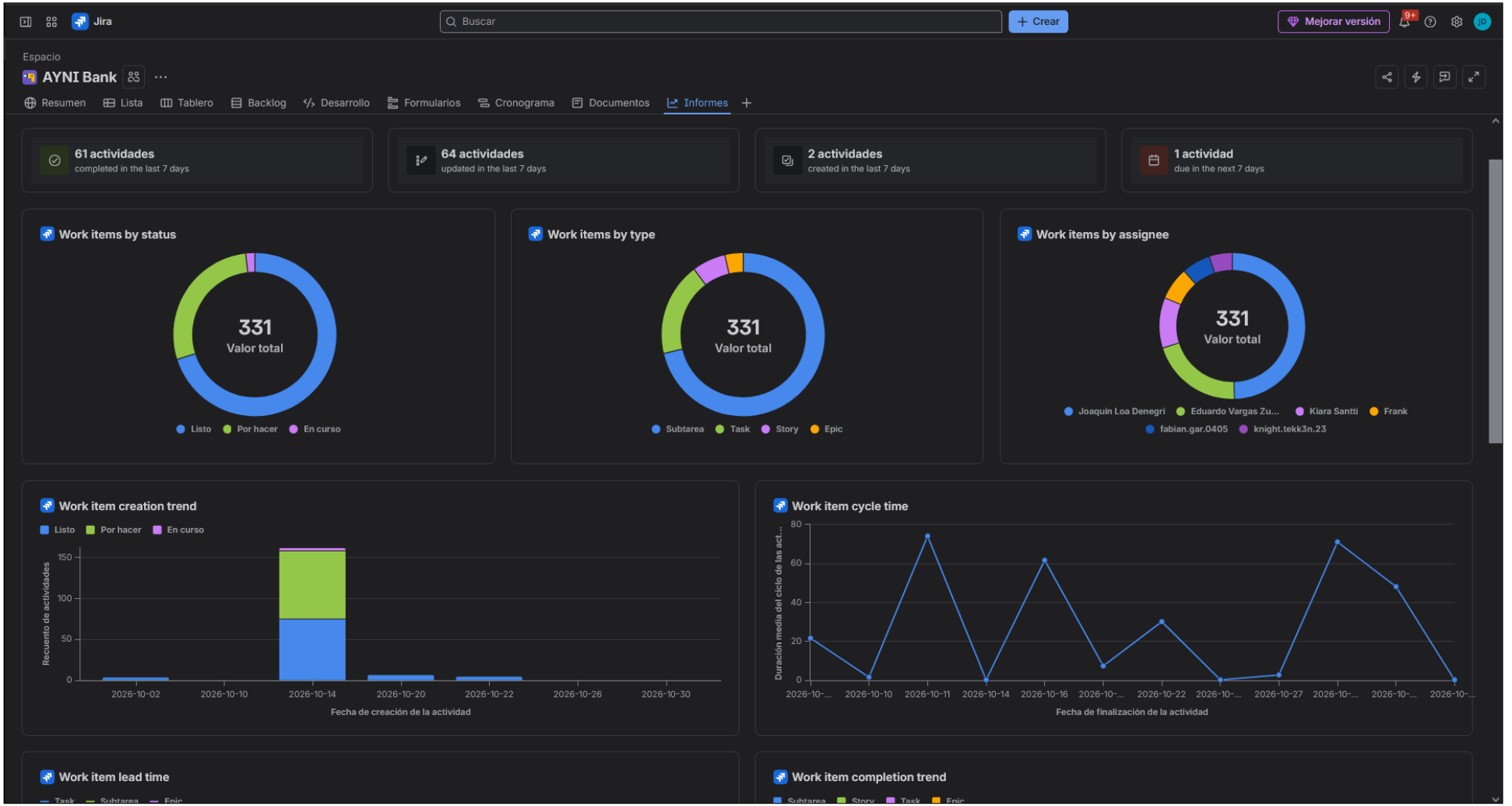
Nota. Captura del tablero público https://load-13.github.io/ayni-tablero/.

Figura 45 · Tablero de Jira del Sprint 5.



Nota. Captura del tablero del Sprint 5 en Jira.

Figura 46 · Informe del Sprint 5 en Jira.



Nota. Captura de la sección Informes del espacio AYNI Bank en Jira.

8. VIDEO DE LAS PRUEBAS AUTOMATIZADAS

Las pruebas también se presentan en un video narrado de 9 minutos (`Ayni-Bank-APF3-video-de-pruebas.mp4`), grabado sobre las ejecuciones reales. Cada capítulo muestra el código de la prueba o su salida junto a lo que comprueba.

Tabla 31 · Capítulos del video

Tiempo	Capítulo	Qué se ve
0:00	Portada	Qué se prueba y las cifras principales
0:14	1. Pruebas funcionales	El código real de la prueba junto al navegador que Playwright controla; la ejecución en la terminal
1:54	2. La API en Swagger	401 sin sesión, ingreso con segundo factor, «Authorize» e idempotencia del depósito
4:05	3. Integración	Testcontainers y los 117 casos, sistema por sistema
5:46	4. Usabilidad	Los cuatro subatributos de ISO/IEC 25010 sobre la aplicación y axe-core
6:42	5. La versión 2 en AWS	La consola de AWS y la aplicación desplegada, en vivo
7:58	6. Resultados	JaCoCo, Playwright, SonarCloud y la CI en GitHub Actions

Nota. Elaboración propia.

9. TECNOLOGÍAS UTILIZADAS EN CADA PUNTO

Tabla 32 · Tecnologías por artefacto

Artefacto	Tecnologías
Pruebas funcionales	JUnit 5, AssertJ, Mockito, Cucumber (Gherkin), Playwright, Vitest, pytest
Cobertura y calidad	JaCoCo, coverage.py, Vitest V8, SonarCloud
Integración	Testcontainers (PostgreSQL, Redis, MinIO), contratos OpenAPI, Spring Boot Test
Usabilidad	Playwright, axe-core (WCAG 2.1 AA), Testing Library
API	Swagger UI (springdoc-openapi), solo en local
Despliegue	GitHub Actions, Amazon ECR, EC2, RDS, S3, SES, Systems Manager, Caddy
Gestión	Jira, GitHub (GitFlow, pull requests), Conventional Commits

Nota. Elaboración propia.

10. CONCLUSIONES

- La versión 2.0.0 está en producción en AWS y se validó contra la nube con pruebas automatizadas.
- **Calidad funcional alta**: 14 de 15 historias funcionales entregadas, 66 de 66 criterios de aceptación ejecutables y 1 411 ejecuciones de prueba sin fallos.
- **Interoperabilidad comprobada** con siete sistemas externos: 117 de 117 casos, tres de ellos contra servicios reales en contenedores.
- **Usabilidad alta** en las cuatro subcaracterísticas de ISO/IEC 25010, con accesibilidad WCAG 2.1 AA.
- Pendientes con fecha: la cuenta en dólares (HU-12, Sprint 6), RabbitMQ con broker real y el cuestionario SUS con usuarios en el proyecto final.